

Classification

Dr. Vimal K. Shrivastava
Assistant Professor (II)
School of Electronics Engineering
KIIT Deemed to be University,
Bhubaneswar

Classification

- In classification, the data is categorized under different labels (classes).
- Here, we want to know which class the input belongs to.
- **Output is categorical (discrete value).**
- **Examples:**
- **Spam mail filtering:** Classifies the mails in regular or spam.
- **Digit recognition service:** Classifies the digit image into 0 to 9.

Classification Techniques

- There are many classification techniques:
 - Logistic Regression
 - K-Nearest Neighbors
 - Decision Tree
 - Support Vector Machine
 - Artificial Neural Network

Logistic Regression

- Logistic Regression is an extension of Linear regression where the **dependent variable is categorical** and not continuous.
- It predicts the output of a categorical variable, which is discrete in nature.
- Hence, it is a classification algorithm and a **supervised machine learning technique**.
- It can be either Yes or No, 0 or 1, true or False, etc. *i.e.* binary outcome.
- However, instead of giving the exact value as 0 and 1, it gives the output as the probability which lies between 0 and 1.

- The logistic regression model transforms the continuous output of linear regression into categorical output using a **sigmoid function**.
- It maps any real-valued set of independent variables input into a value between 0 and 1.

- Let the independent input features be:
$$X = \begin{bmatrix} x_{11} & \dots & x_{1m} \\ x_{21} & \dots & x_{2m} \\ \vdots & \ddots & \vdots \\ x_{n1} & \dots & x_{nm} \end{bmatrix}$$

- The dependent variable is Y having only binary value *i.e.* 0 or 1.

$$Y = \begin{cases} 0 & \text{if } \textit{Class 1} \\ 1 & \text{if } \textit{Class 2} \end{cases}$$

- Now, apply the linear function to the input variables X:

$$z = (\sum_{i=1}^n w_i x_i) + b$$

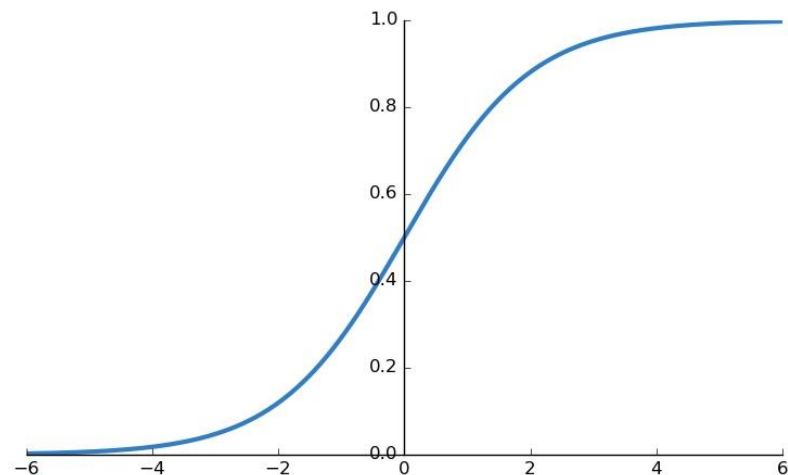
Here x_i is the i th observation of X, $w_i = [w_1, w_2, w_3, \dots, w_m]$ is the weights or Coefficient, and b is the bias term also known as intercept. simply this can be represented as the

$$z = w \cdot X + b$$

- Now we use the sigmoid function where the input will be z and we find the probability between 0 and 1. *i.e.* predicted y .

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma(z) = \frac{1}{1 + e^{-(w \cdot X + b)}}$$



- As shown in the figure sigmoid function converts the continuous variable data into the probability i.e. between 0 and 1.

$$\sigma(z)$$

- tends towards 1 as

$$z \rightarrow \infty$$

$$\sigma(z)$$

- tends towards 0 as

$$z \rightarrow -\infty$$

$$\sigma(z)$$

- is always bounded between 0 and 1

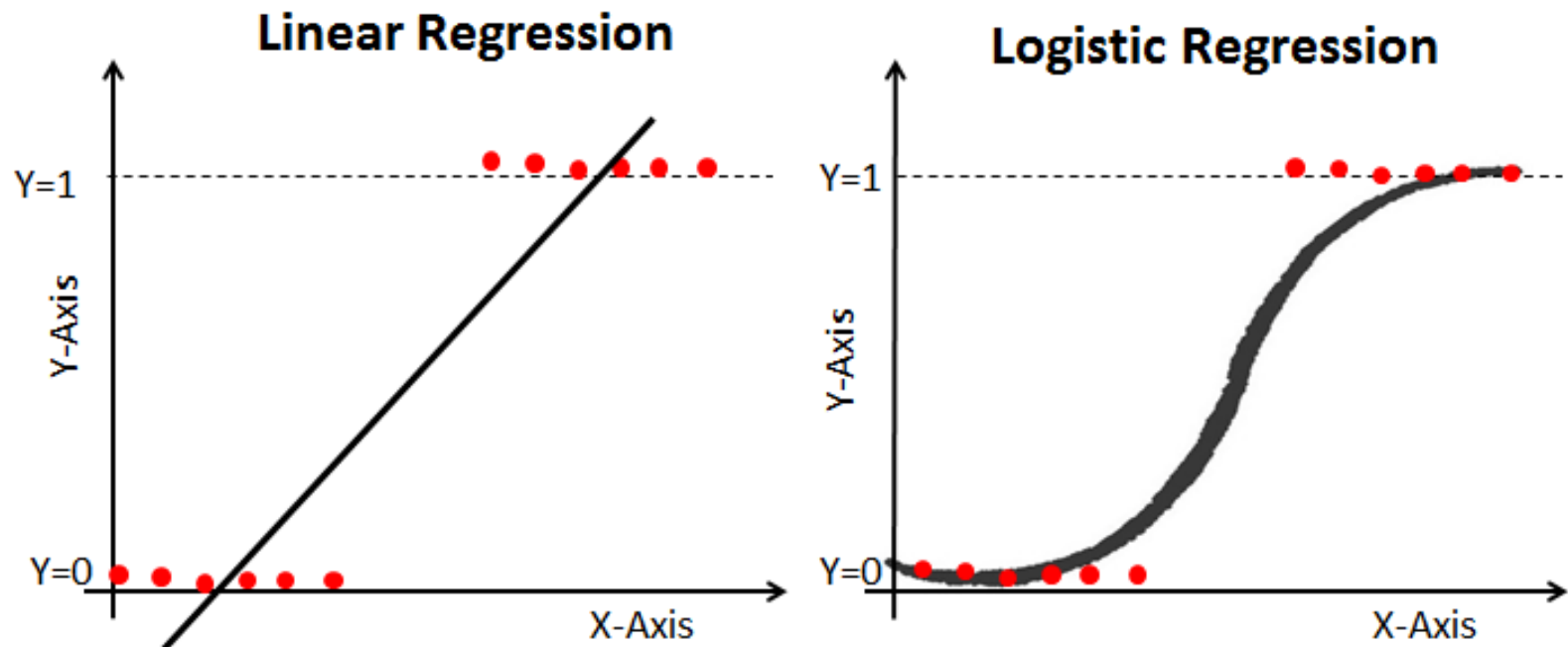
- where the probability of being a class can be measured as:

$$P(y = 1) = \sigma(z)$$

$$P(y = 0) = 1 - \sigma(z)$$

- Consider the following example where the data points belong to one of the two categories:
- A threshold (usually 0.5) is then used to categorize the data into one of the two categories.

$$\text{decision}(x) = \begin{cases} 1 & \text{if } P(y=1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$



Training of Logistic Regression Model

$$\text{Loss} = -\frac{1}{\text{output size}} \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i + (1 - y_i) \cdot \log (1 - \hat{y}_i)$$

- It is known as **log-loss** or **binary cross-entropy** loss function.
- To find the coefficients (weights) that minimize the loss function we will use Gradient Descent.

$$W_j = W_j - L \frac{\partial(\text{Loss})}{\partial W_j}$$

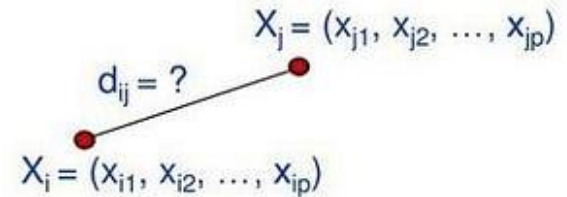
K-Nearest Neighbor (KNN)

- It is one of the simplest and widely used classification algorithm.
- It is based on **Supervised Learning** technique.
- It relies on the idea that similar data points tend to have similar class labels.
- KNN predicts the class label of a new data point by considering its K closest neighbors in the training dataset.

KNN Algorithm

- **Step 1:** Divide the dataset into training and test set.
- **Step 2:** Choose the value of K i.e. the nearest data points. K should be integer.
- **Step 3:** For each data point in the test set do the following:
 - **3.1:** Calculate the distance between test data and each training data using Euclidean distance.
 - **3.2:** Now, based on the distance value, sort them in ascending order.
 - **3.3:** Next, it will choose the top K training data from the sorted array.
 - **3.4:** Now, it will assign a class to the test data based on most frequent class of these rows.

Distance Metrics



- **Minkowski distance**

$$d(i, j) = \sqrt[q]{|x_{i1} - x_{j1}|^q + |x_{i2} - x_{j2}|^q + \dots + |x_{ip} - x_{jp}|^q}$$

- **Euclidean distance**

$$q = 2$$

$$d(i, j) = \sqrt{|x_{i1} - x_{j1}|^2 + |x_{i2} - x_{j2}|^2 + \dots + |x_{ip} - x_{jp}|^2}$$

- **Manhattan distance**

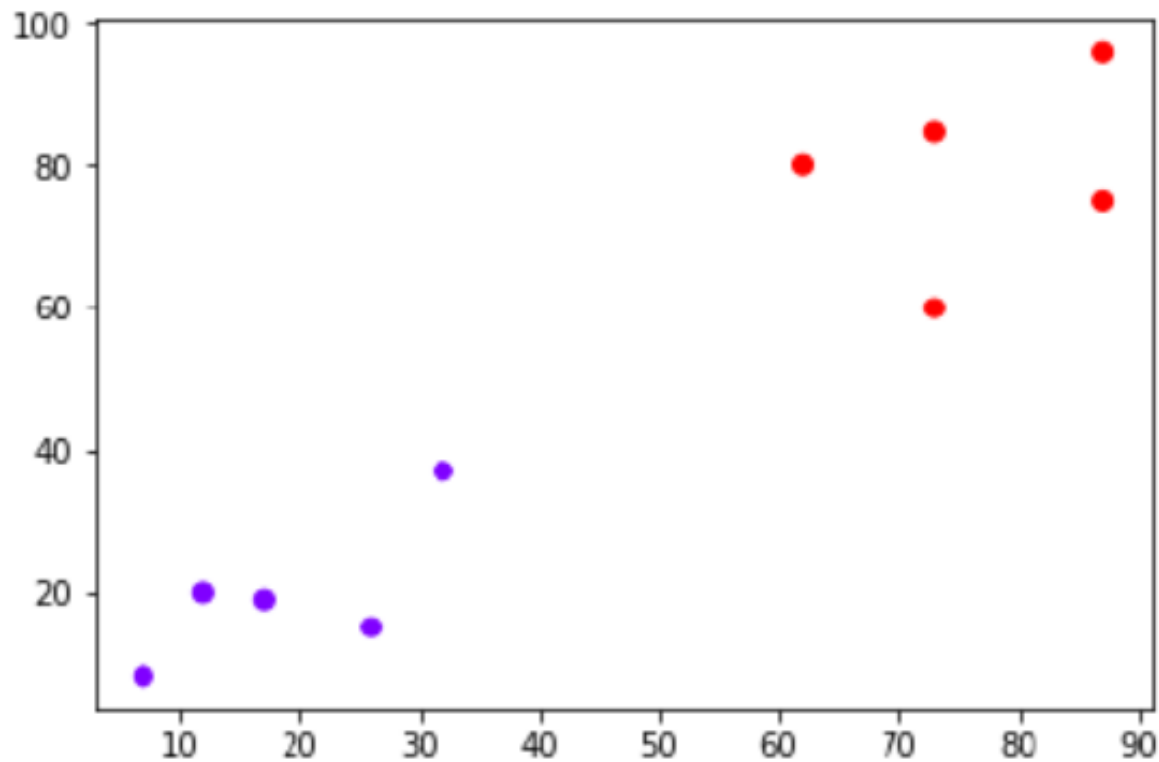
$$q = 1$$

$$d(i, j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \dots + |x_{ip} - x_{jp}|$$

- Usually, we use the **Euclidean approach**, which is the most widely used distance measure

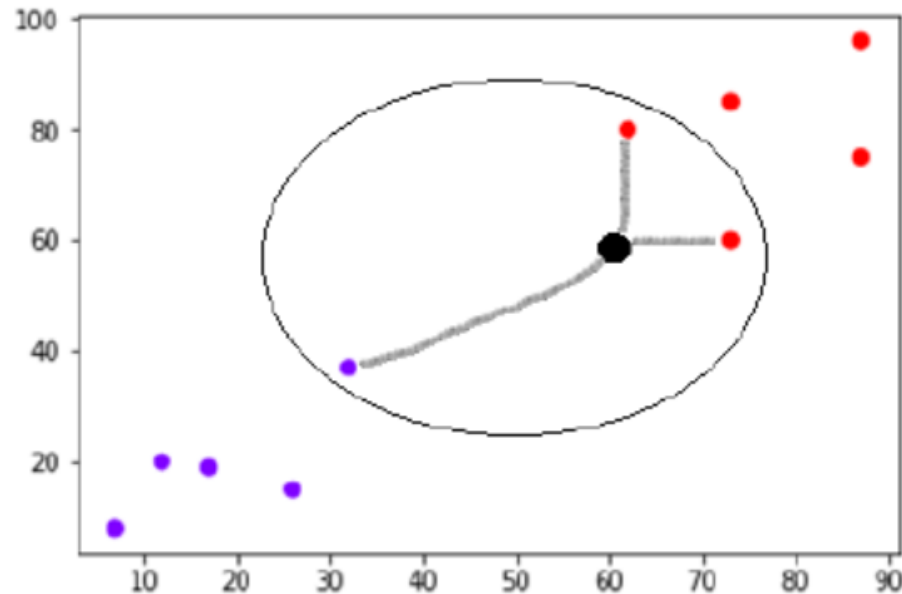
KNN Example

- Suppose we have a dataset which can be plotted as follows:



KNN Example

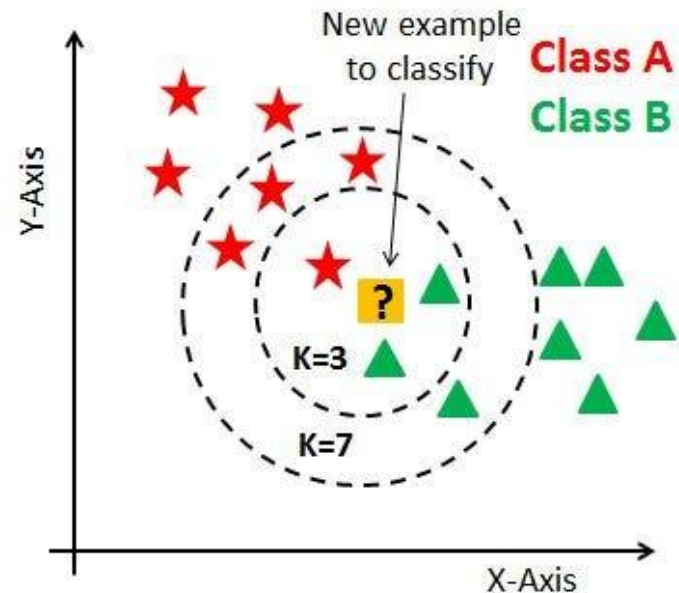
- Now, we need to classify new data point with black dot into blue or red class. We are assuming $K = 3$ i.e. it would find three nearest data points.



- We can see in the above diagram that the three nearest neighbors of the data point with black dot. Among those three, two of them lies in Red class hence the black dot will also be assigned in red class.

Determining the Optimal K

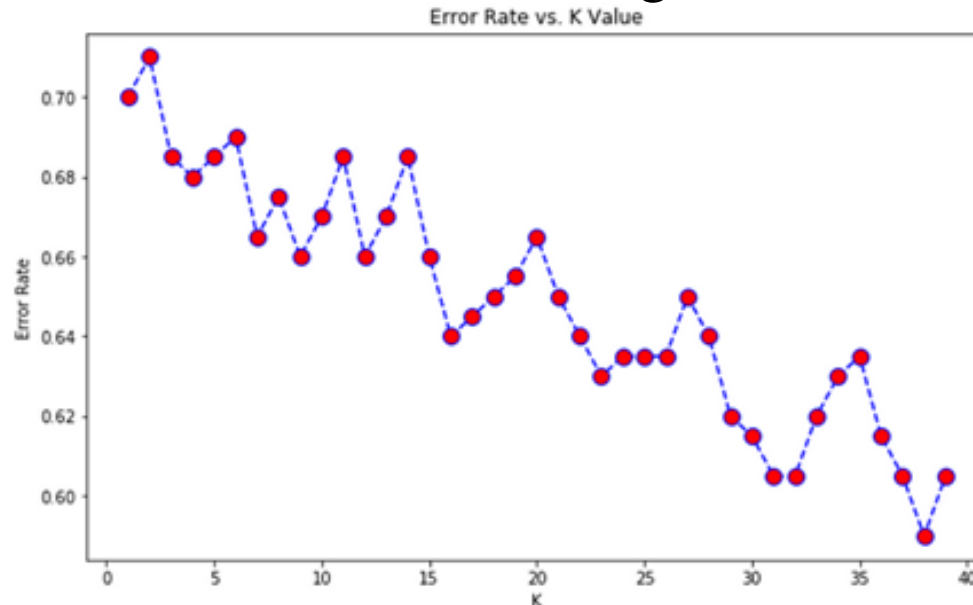
- As you can verify from the figure, if we proceed with $K=3$, then we predict that test input belongs to class B, and if we continue with $K=7$, then we predict that test input belongs to class A.
- That's how you can imagine that the K value has a powerful effect on KNN performance.



- Choosing the right K value is important for getting accurate results with the algorithm. Two cases could arise:-
 1. The K value is too less, and it leads to **underfitting** if the K value is too less, i.e., if we select $K=1$ or $K=2$, we would be only looking at a very small number of neighbors and hence won't be utilizing the full scope of the data.
 2. The K value is too large, which may lead to **overfitting**. If the K value is too large, we might consider a lot of outliers, which would lead to inaccurate results.
- One way to find the optimal K value is the **square root of N**, where N is the total number of samples.
- Another way is: Elbow method.

Elbow Method

- Initialize a random K value and start computing.
- Draw a plot between error rate and K denoting values in a defined range.
- Then choose the K value as having a minimum error rate.



- From the plot, you can see that the smallest error we got is 0.59 at $K=37$.

Pros and Cons of KNN

Pros

1. Easy to understand.
2. No assumptions about data.
3. Can be applied to both classification and regression.
4. Works easily on multi-class problems.

Cons

1. Memory Intensive / Computationally expensive.
2. Sensitive to scale of data.
3. Not work well on highly imbalanced dataset.
4. Struggle when high number of independent variables.

Problem 01:

We have data from the questionnaires survey (to ask people opinion) and objective testing with two attributes (acid durability and strength) to classify whether a special paper tissue is good or not. Here is four training samples

X1 = Acid Durability (seconds)	X2 = Strength	Y = Classification
	(kg/square meter)	
7	7	Bad
7	4	Bad
3	4	Good
1	4	Good

Now the factory produces a new paper tissue that pass laboratory test with $X1 = 3$ and $X2 = 7$. Without another expensive survey, can we guess what the classification of this new tissue is?

Solution

1. Determine parameter K = number of nearest neighbors

Suppose use $K = 3$

2. Calculate the distance between the query-instance and all the training samples

Coordinate of query instance is (3, 7), instead of calculating the distance we compute square distance which is faster to calculate (without square root)

X1 = Acid Durability (seconds)	X2 = Strength (kg/square meter)	Square Distance to query instance (3, 7)
7	7	$(7-3)^2 + (7-7)^2 = 16$
7	4	$(7-3)^2 + (4-7)^2 = 25$
3	4	$(3-3)^2 + (4-7)^2 = 9$
1	4	$(1-3)^2 + (4-7)^2 = 13$

3. Sort the distance and determine nearest neighbors based on the K-th minimum distance

X1 = Acid Durability (seconds)	X2 = Strength (kg/square meter)	Square Distance to query instance (3, 7)	Rank minimum distance	Is it included in 3-Nearest neighbors?
7	7	$(7-3)^2 + (7-7)^2 = 16$	3	Yes
7	4	$(7-3)^2 + (4-7)^2 = 25$	4	No
3	4	$(3-3)^2 + (4-7)^2 = 9$	1	Yes
1	4	$(1-3)^2 + (4-7)^2 = 13$	2	Yes

4. Gather the category Y of the nearest neighbors. Notice in the second row last column that the category of nearest neighbor (Y) is not included because the rank of this data is more than 3 (=K).

X1 = Acid Durability (seconds)	X2 = Strength (kg/square meter)	Square Distance to Rank query instance (3, 7) minimum distance	Is it included in 3-Nearest neighbors?	Y = Category of nearest Neighbor
7	7	$(7-3)^2 + (7-7)^2 = 16$ 3	Yes	Bad
7	4	$(7-3)^2 + (4-7)^2 = 25$ 4	No	-
3	4	$(3-3)^2 + (4-7)^2 = 9$ 1	Yes	Good
1	4	$(1-3)^2 + (4-7)^2 = 13$ 2	Yes	Good

5. Use simple majority of the category of nearest neighbors as the prediction value of the query instance

We have 2 good and 1 bad, since $2 > 1$ then we conclude that a new paper tissue that pass laboratory test with X1 = 3 and X2 = 7 is included in **Good** category.

Classification Model Evaluation

Confusion Matrix

- A confusion matrix is a tabular way of visualizing the performance of the model.
- A Confusion matrix is an $N \times N$ matrix used for evaluating the performance of a classification model, where N is the number of target classes.
- The matrix compares the actual target values with those predicted by the machine learning model.
- This gives us a holistic view of how well our classification model is performing and what kinds of errors it is making.
- For a binary classification (two class problem), we would have a 2×2 matrix as shown below with 4 values:

- **Confusion Matrix for Binary Classification:**

		Predicted Class	
		Positive	Negative
Actual Class	Positive	TP	FN
	Negative	FP	TN

- **True Positive (TP):** It refers to the number of predictions where the classifier correctly predicts the positive class as positive.
- **True Negative (TN):** It refers to the number of predictions where the classifier correctly predicts the negative class as negative.
- **False Positive (FP):** It refers to the number of predictions where the classifier incorrectly predicts the negative class as positive. It is also known as **Type-I error**.
- **False Negative (FN):** It refers to the number of predictions where the classifier incorrectly predicts the positive class as negative. It is also known as **Type-II error**.

Problem 1:

		ACTUAL VALUES	
		POSITIVE	NEGATIVE
PREDICTED VALUES	POSITIVE	560	60
	NEGATIVE	50	330

- Find TP, TN, FP and FN.
- **Solution:**
- True Positive (TP) = 560; meaning 560 positive class data points were correctly classified by the model.
- True Negative (TN) = 330; meaning 330 negative class data points were correctly classified by the model.
- False Positive (FP) = 60; meaning 60 negative class data points were incorrectly classified as belonging to the positive class by the model.
- False Negative (FN) = 50; meaning 50 positive class data points were incorrectly classified as belonging to the negative class by the model.

Performance Measures from the Confusion Matrix

- **Accuracy:** It gives the overall accuracy of the model, meaning the fraction of the total samples that were correctly classified by the classifier.
- **Accuracy = $(TP+TN)/(TP+TN+FP+FN)$**
- **Misclassification Rate:** It tells that what fraction of predictions were incorrect. It is also known as Classification Error.
- **Misclassification Rate = $(FP+FN)/(TP+TN+FP+FN)$ or**
- **Misclassification Rate = $(1-Accuracy)$**

Why Do We Need a Confusion Matrix?

Why Do We Need Other Parameters?

- Let's say there are two values for our target variable: Sick and Not Sick.
- Our dataset is an example of an **imbalanced dataset**. There are 947 data points for the negative class (Not Sick) and 53 data points for the positive class (Sick).

ID	Actual Sick?	Predicted Sick?	Outcome
1	1	1	TP
2	0	0	TN
3	0	0	TN
4	1	1	TP
5	0	0	TN
6	0	0	TN
7	1	0	FN
8	0	1	FP
9	0	0	TN
10	1	0	FN
1000	0	0	TN

- Let's say the total outcome values are:

$$\begin{aligned} TP &= 30, TN = 930, \\ FP &= 30, FN = 10 \end{aligned}$$

- So, the accuracy of our model turns out to be:

$$Accuracy = \frac{30 + 930}{30 + 30 + 930 + 10} = 0.96$$

- 96%! Not bad!

- But it gives the wrong idea about the result for an **imbalanced dataset**.
- The model is saying, “I can predict accurately 96% of the time”. However, it is unable to predict correctly the 10 sick cases (FN=10).
- This is where we come across the confusion matrix and other performance parameters like Precision, Recall and F1-Score.

- **Precision:** It tells that what fraction of predictions as a positive class were actually positive.
- Precision is a useful metric in cases where False Positive is a higher concern than False Negatives.
- Precision is important in music or video recommendation systems, e-commerce websites, etc. Wrong results could lead to customer churn and be harmful to the business.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

- **Recall:** It tells that what fraction of all positive samples were correctly predicted as positive by the classifier. It is also known as **True Positive Rate (TPR) or Sensitivity**.
- Recall is a useful metric in cases where False Negative is a higher concern than False Positive.
- Recall is important in medical cases where it doesn't matter whether we raise a false alarm, but the actual positive cases should not go undetected!

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

		ACTUAL VALUES	
		POSITIVE	NEGATIVE
PREDICTED VALUES	POSITIVE	TP (30)	FP (30)
	NEGATIVE	FN (10)	TN (930)

Sick people correctly predicted as sick by the model

Healthy people incorrectly predicted as sick by the model

Sick people incorrectly predicted as not sick by the model

Healthy people correctly predicted as not sick by the model

$$Precision = \frac{30}{30 + 30} = 0.5$$

$$Recall = \frac{30}{30 + 10} = 0.75$$

- 50% percent of the correctly predicted cases turned out to be positive cases.
- Whereas 75% of the positives were successfully predicted by our model.

- But there will be cases where there is no clear distinction between whether Precision is more important or Recall. What should we do in those cases? We combine them!
- In practice, when we try to increase the precision of our model, the recall goes down, and vice-versa.
- The **F1-score** captures both the trends in a single value.

$$F_1\text{-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

- **Specificity:** It tells that what fraction of all negative samples are correctly predicted as negative by the classifier. It is also known as **True Negative Rate (TNR)**.

$$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$$

Confusion Matrix for Multi-Class Classification

- For simplicity, let's consider our multi-class classification problem to be a 3-class classification problem.
- Say, we have a dataset that has three class labels, namely Apple, Orange and Mango.

		True Class		
		Apple	Orange	Mango
Predicted Class	Apple	7	8	9
	Orange	1	2	3
	Mango	3	2	1

- We have to find TP, TN, FP and FN for each individual class.
- For example, if we take class Apple, then let's see what are the values of the metrics from the confusion matrix.

- $TP = 7$
- $TN = (2+3+2+1) = 8$
- $FP = (8+9) = 17$
- $FN = (1+3) = 4$

		True Class		
		Apple	Orange	Mango
Predicted Class	Apple	7	8	9
	Orange	1	2	3
	Mango	3	2	1

- Now, we can calculate the performance measures for class Apple.
- $Precision = 7/(7+17) = 0.29$
- $Recall = 7/(7+4) = 0.64$

- Similarly, we can calculate the measures for the other classes.

Class	Precision	Recall	F1-score
Apple	0.29	0.64	0.40
Orange	0.33	0.17	0.22
Mango	0.17	0.08	0.11

- **Problem 2:**

n=165	Predicted: NO	Predicted: YES
Actual: NO	50	10
Actual: YES	5	100

- Find TP, TN, FP and FN and other performance parameters.

- **Solution:**

- TP=100; TN=50; FP=10; FN=5

- **Accuracy:** Overall, how often is the classifier correct?

$$(TP+TN)/total = (100+50)/165 = 0.91$$

- **Misclassification Rate:** Overall, how often is it wrong?

$$(FP+FN)/total = (10+5)/165 = 0.09$$

- Equivalent to 1 minus Accuracy also known as "Error Rate"

- **True Positive Rate:** When it's actually yes, how often does it predict yes?
 - $TP/\text{actual yes} = 100/105 = 0.95$
 - also known as "Sensitivity" or "Recall"
- **True Negative Rate:** When it's actually no, how often does it predict no?
 - $TN/\text{actual no} = 50/60 = 0.83$
 - also known as "Specificity"
- **False Positive Rate:** When it's actually no, how often does it predict yes?
 - $FP/\text{actual no} = 10/60 = 0.17$
 - equivalent to 1 minus True Negative Rate
- **Precision:** When it predicts yes, how often is it correct?
 - $TP/\text{predicted yes} = 100/110 = 0.91$

- **Problem 3:**
- Define the following terms associated with confusion matrix.
 - i. Accuracy
 - ii. Misclassification
 - iii. Precision
 - iv. Sensitivity
 - v. Specificity
- Thus, compute these metrics for the confusion matrix given below.

		Actual	
		True	False
Predicted	True	45	55
	False	5	395

Receiver Operating Characteristics (ROC) Curve

- It is one of the most important evaluation metrics for checking any classification model's performance.
- An ROC curve is a graph showing the performance of a classification model at all classification thresholds.
- By looking at this graph, we can understand how good the model is and choose the threshold that gives us the right balance between correct and incorrect predictions.
- The ROC curve is plotted with True Positive Rate (TPR) against the False Positive Rate (FPR) where TPR is on the y-axis and FPR is on the x-axis.

$$\text{TPR / Recall / Sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

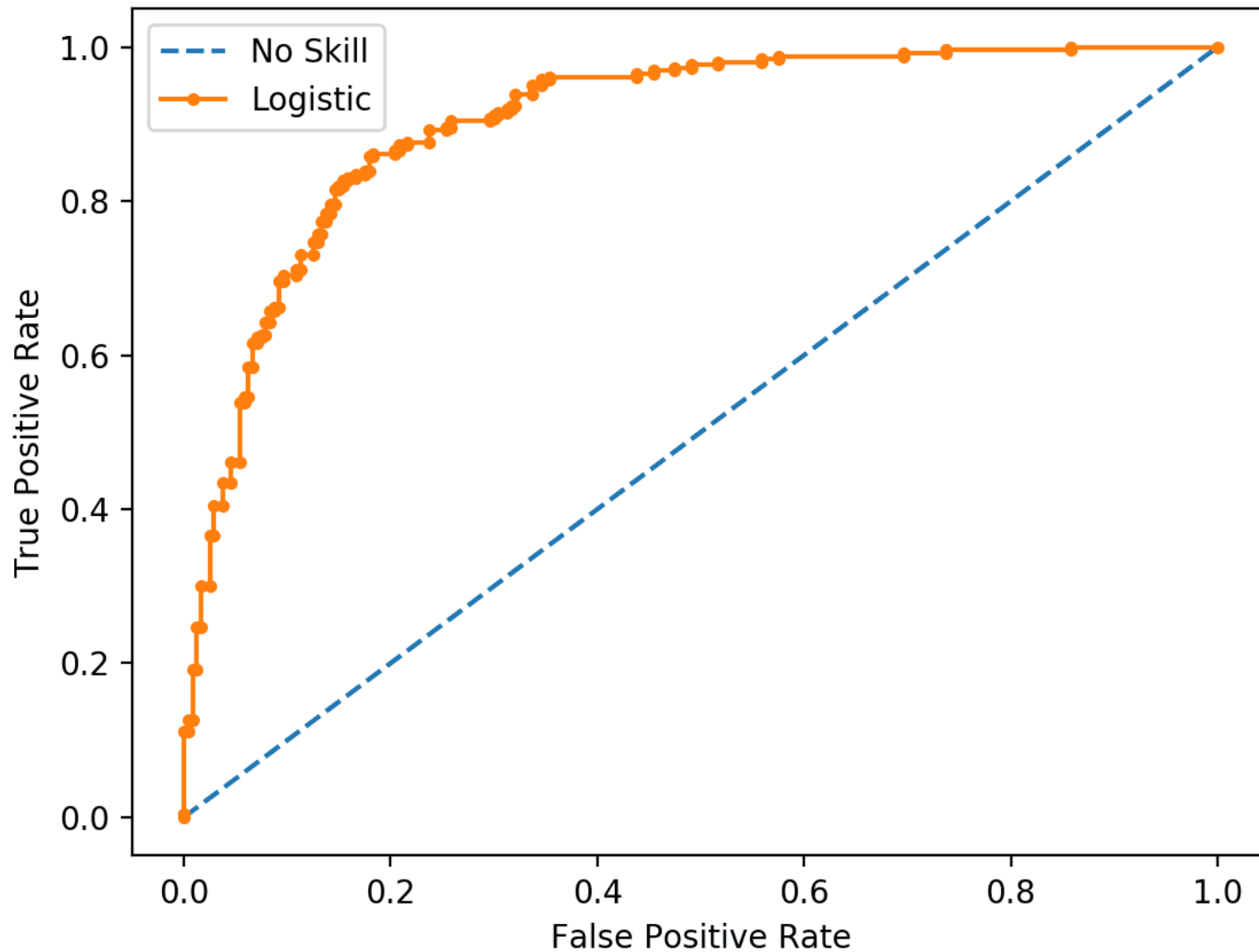
$$\text{FPR} = 1 - \text{Specificity}$$

$$= \frac{\text{FP}}{\text{TN} + \text{FP}}$$

- Have a look at the table below:

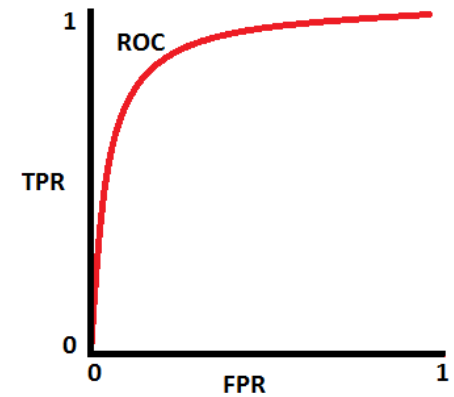
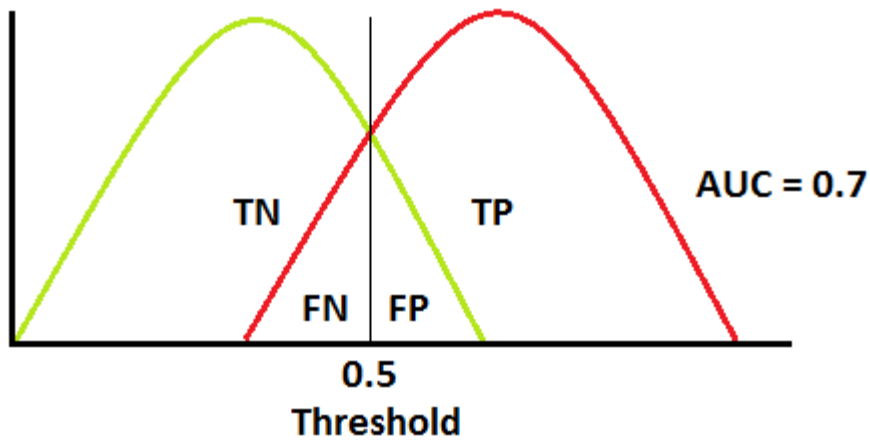
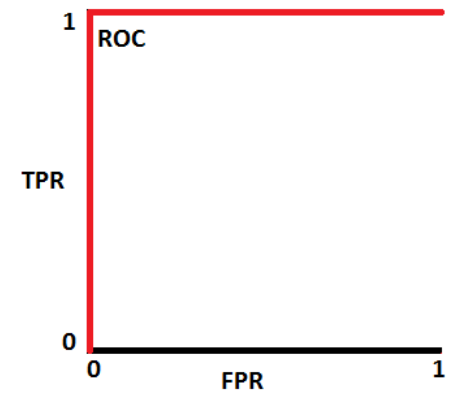
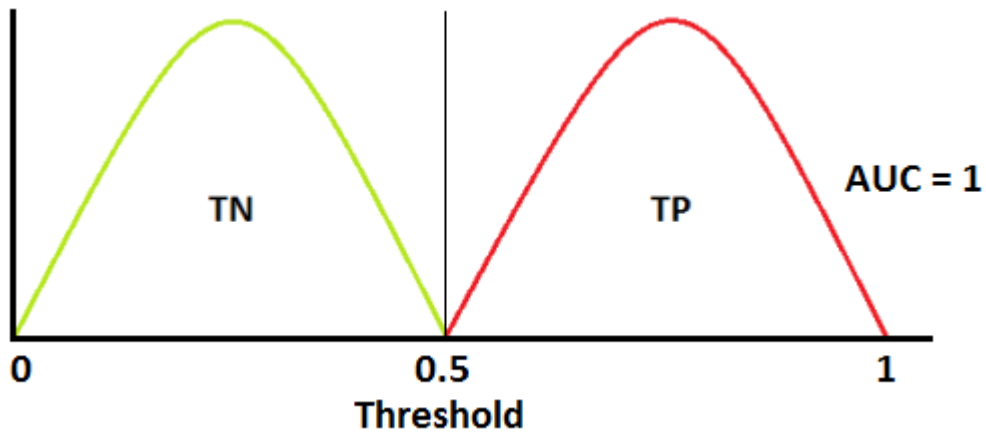
ID	Actual	Prediction Probability	>0.6	>0.7	>0.8	Metric
1	0	0.98	1	1	1	
2	1	0.67	1	0	0	
3	1	0.58	0	0	0	
4	0	0.78	1	1	0	
5	1	0.85	1	1	1	
6	0	0.86	1	1	1	
7	0	0.79	1	1	0	
8	0	0.89	1	1	1	
9	1	0.82	1	1	1	
10	0	0.86	1	1	1	
			0.75	0.5	0.5	TPR
			1	1	0.66	FPR
			0	0	0.33	TNR
			0.25	0.5	0.5	FNR

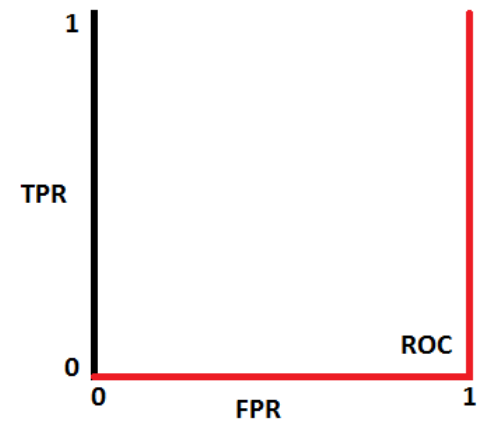
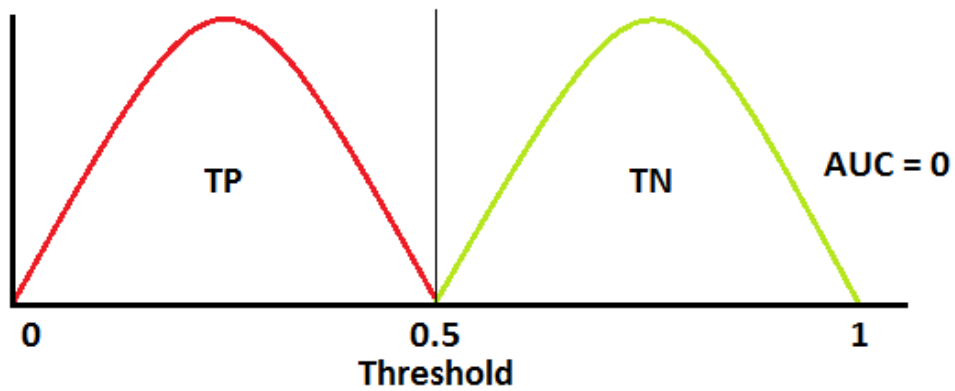
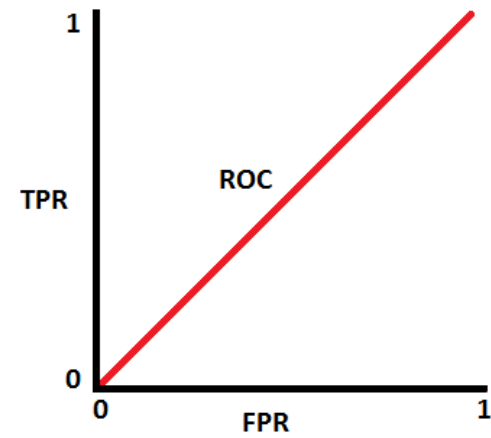
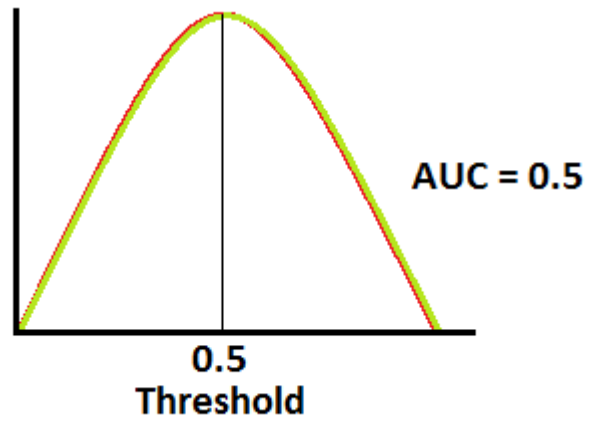
- The metrics change with the changing threshold values.



Area Under The ROC Curve (AUC)

- AUC represents the degree or measure of separability.
- Higher the AUC, the better the model is at predicting 0 classes as 0 and 1 classes as 1.
- **Ex:** The Higher the AUC, the better the model is at distinguishing between patients with the disease and no disease.
- An excellent model has AUC near to the 1 which means it has a good measure of separability.
- A poor model has an AUC near 0 which means it has the worst measure of separability.





Naïve Bayes Classifier

- A Naive Bayes classifier is a probabilistic machine learning model that's used for classification task.
- It is based on Bayes' Theorem.
- The assumption made here is that the features are independent. That is presence of one particular feature does not affect the other. Hence it is called naive.
- Naive Bayes model is easy to build and particularly useful for very large data sets.

Bayes' Theorem

- In probability theory , Bayes' theorem explains the probability of most probable event, based on prior given conditions that might be related to the event.
- For example, if any disease is related to age, then, using Bayes' theorem, a person's age can be used to more accurately assess the probability that they have similar disease, compared to the judgement of the probability of disease made without knowledge of the person's age.
- Bayes' theorem is used to compute the conditional probability of an event using prior knowledge.

Bayes Theorem Derivation

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

- **Proof:**
- From the definition of **conditional probability**:
- $P(A | B) = P(A \cap B) / P(B)$, where $P(B) \neq 0$
- $P(B | A) = P(B \cap A) / P(A)$, where $P(A) \neq 0$
- Here, the joint probability $P(A \cap B)$ of both events A and B being true such that,
- $P(B \cap A) = P(A \cap B)$
- $P(A \cap B) = P(A | B) P(B) = P(B | A) P(A)$

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

LIKELIHOOD

The probability of "B" being True, given "A" is True

PRIOR

The probability "A" being True. This is the knowledge.

The diagram illustrates the components of Bayes' theorem. At the top left, 'LIKELIHOOD' is defined as the probability of 'B' being True given 'A' is True. A yellow arrow points from this definition to the numerator term $P(B|A)$ in the equation. At the top right, 'PRIOR' is defined as the probability 'A' being True, which is the knowledge. A yellow arrow points from this definition to the numerator term $P(A)$ in the equation. The equation itself is $P(A|B) = \frac{P(B|A).P(A)}{P(B)}$. At the bottom left, 'POSTERIOR' is defined as the probability of 'A' being True given 'B' is True. A yellow arrow points from this definition to the left side of the equation, $P(A|B)$. At the bottom right, 'MARGINALIZATION' is defined as the probability 'B' being True. A yellow arrow points from this definition to the denominator term $P(B)$ in the equation.

$$P(A|B) = \frac{P(B|A).P(A)}{P(B)}$$

POSTERIOR

The probability of "A" being True, given "B" is True

MARGINALIZATION

The probability "B" being True.

- Bayes' theorem is stated mathematically as the following equation:

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

- where,
- A and B are events and $P(B) \neq 0$.
- $P(A|B)$ is conditional probability: the likelihood of event A occurring given that B is true.
- $P(B|A)$ is also a conditional probability :the likelihood of event B occurring given that A is true.
- $P(A)$ and $P(B)$ are probabilities of observing A and B independently, this is known as marginal probability.

- **Example:**
- **Step 1: (A) Find Individual probabilities:**
- Suppose, we have 2 boxes A and B, then we have to calculate the probability of selecting the boxes.
- It is simple, since we have to select one box out two so:

$$P(A) = 1/2 \text{ and } P(B) = 1/2$$

- What if individual probabilities are given:
- **$P(A) = 60\%$, $P(B) = 30\%$ and $P(C) = 10\%$**
- **$P(A) = 0.6$, $P(B) = 0.3$ and $P(C) = 0.1$**

- **Step 2 : (B) Find conditional probability:**
- It is represented as $P(\mathbf{x} | \mathbf{A})$ here $\mathbf{x}$ is selecting element from set of $\mathbf{A}$.
- Suppose, we have a box contains 5 red and 3 white balls.
- Calculate the probability of selecting red ball from the box.
- $P(R | A) = (\text{red ball}) / (\text{total number of balls}) = 5/8$

Naive Bayes Classifier

- **Example:**
- Consider the weather dataset. The dataset is represented as below.

	OUTLOOK	TEMPERATURE	HUMIDITY	WINDY	PLAY GOLF
0	Rainy	Hot	High	False	No
1	Rainy	Hot	High	True	No
2	Overcast	Hot	High	False	Yes
3	Sunny	Mild	High	False	Yes
4	Sunny	Cool	Normal	False	Yes
5	Sunny	Cool	Normal	True	No
6	Overcast	Cool	Normal	True	Yes
7	Rainy	Mild	High	False	No
8	Rainy	Cool	Normal	False	Yes
9	Sunny	Mild	Normal	False	Yes
10	Rainy	Mild	Normal	True	Yes
11	Overcast	Mild	High	True	Yes
12	Overcast	Hot	Normal	False	Yes
13	Sunny	Mild	High	True	No

- We have to classify whether the day is suitable for playing golf, given the features of the day.
- According to this example, Bayes theorem can be rewritten as:

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)}$$

- The variable y is the class variable (play golf), which represents if it is suitable to play golf or not given the conditions.
- Variable X represents the features which is given as:

$$X = (x_1, x_2, x_3, \dots, x_n)$$

- Here $x_1, x_2, \dots, x_n$ represent the features, i.e. they can be mapped to outlook, temperature, humidity and windy.

- By substituting for X and expanding using the chain rule we get:

$$P(y|x_1, \dots, x_n) = \frac{P(x_1|y)P(x_2|y)\dots P(x_n|y)P(y)}{P(x_1)P(x_2)\dots P(x_n)}$$

- Now, you can obtain the values for each by looking at the dataset and substitute them into the equation.
- For all entries in the dataset, the denominator does not change, it remain static. Therefore, the denominator can be removed and a proportionality can be introduced.

$$P(y|x_1, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i|y)$$

- In this example, the class variable ($\mathbf{y}$) has only two outcomes, yes or no.
- There could be cases where the classification could be multivariate. Therefore, we need to find the class $\mathbf{y}$ with maximum probability.

$$y = \operatorname{argmax}_y P(y) \prod_{i=1}^n P(x_i|y)$$

- Using the above function, we can obtain the class, given the features.

- Let us apply the above formula manually on weather dataset. For this, we need to do some precomputations on our dataset.

Outlook	Temperature	Humidity	Windy	PlayTennis
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

- We need to find $P(x_i|y_j)$ for each x_i in X and y_j in y . All these calculations have been demonstrated in the tables below:

Play		P(Yes)/P(No)
Yes	9	9/14
No	5	5/14
Total	14	100%

Outlook

	Yes	No	P(yes)	P(no)
Sunny	2	3	2/9	3/5
Overcast	4	0	4/9	0/5
Rainy	3	2	3/9	2/5
Total	9	5	100%	100%

Temperature

	Yes	No	P(yes)	P(no)
Hot	2	2	2/9	2/5
Mild	4	2	4/9	2/5
Cool	3	1	3/9	1/5
Total	9	5	100%	100%

Humidity

	Yes	No	P(yes)	P(no)
High	3	4	3/9	4/5
Normal	6	1	6/9	1/5
Total	9	5	100%	100%

Wind

	Yes	No	P(yes)	P(no)
False	6	2	6/9	2/5
True	3	3	3/9	3/5
Total	9	5	100%	100%

- Let us test it on a new set of features (let us call it today):

today = (Sunny, Hot, Normal, False)

So, probability of playing golf is given by:

$$P(Yes|today) = \frac{P(SunnyOutlook|Yes)P(HotTemperature|Yes)P(NormalHumidity|Yes)P(NoWind|Yes)P(Yes)}{P(today)}$$

and probability to not play golf is given by:

$$P(No|today) = \frac{P(SunnyOutlook|No)P(HotTemperature|No)P(NormalHumidity|No)P(NoWind|No)P(No)}{P(today)}$$

Since, $P(today)$ is common in both probabilities, we can ignore $P(today)$ and find proportional probabilities as:

$$P(Yes|today) \propto \frac{2}{9} \cdot \frac{2}{9} \cdot \frac{6}{9} \cdot \frac{6}{9} \cdot \frac{9}{14} \approx 0.0141$$

and

$$P(No|today) \propto \frac{3}{5} \cdot \frac{2}{5} \cdot \frac{1}{5} \cdot \frac{2}{5} \cdot \frac{5}{14} \approx 0.0068$$

Now, since

$$P(Yes|today) + P(No|today) = 1$$

These numbers can be converted into a probability by making the sum equal to 1 (normalization):

$$P(Yes|today) = \frac{0.0141}{0.0141+0.0068} = 0.67$$

and

$$P(No|today) = \frac{0.0068}{0.0141+0.0068} = 0.33$$

Since

$$P(Yes|today) > P(No|today)$$

So, prediction that golf would be played is 'Yes'.

Pros and Cons of Naive Bayes

- **Pros:**
- It is easy and fast to predict class of test dataset.
- When assumption of independence holds, a Naive Bayes classifier performs better compare to other models like logistic regression and we need less training data.
- It perform well in case of categorical input variables compared to numerical variable(s). For numerical variable, normal distribution is assumed (bell curve, which is a strong assumption).
- **Cons:**
- If categorical variable has a category (in test data set), which was not observed in training data set, then model will assign a 0 (zero) probability and will be unable to make a prediction. This is often known as “Zero Frequency”.
- Another limitation of Naive Bayes is the assumption of independent predictors. In real life, it is almost impossible that we get a set of predictors which are completely independent.

Decision Tree (DT)

Example 1

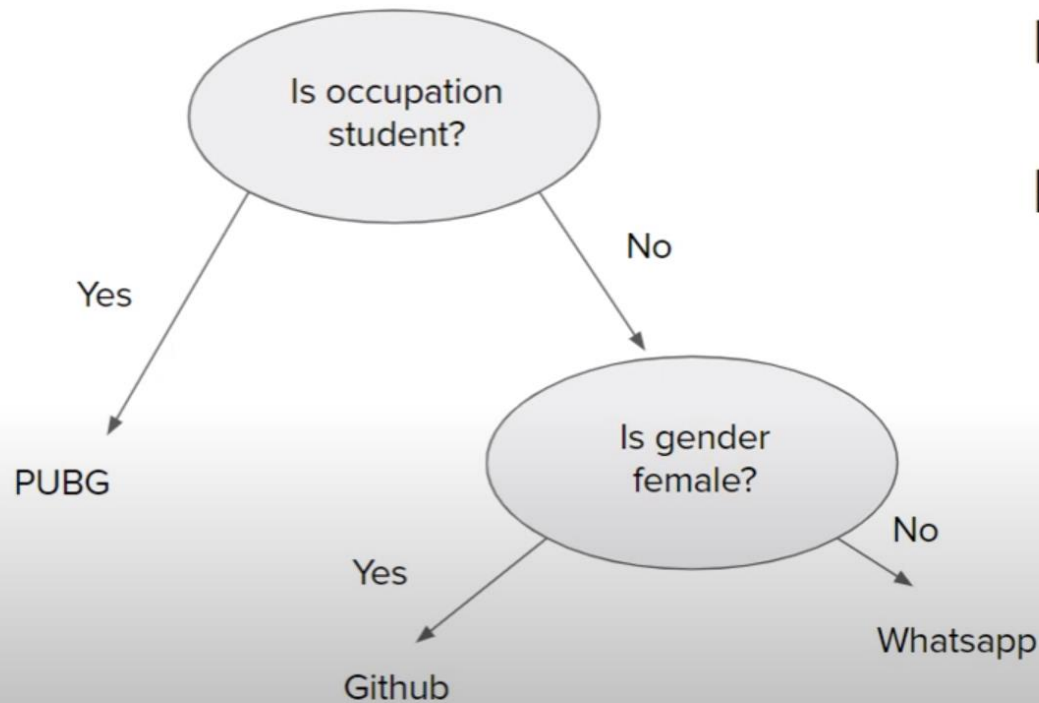
Gender	Occupation	Suggestion
F	Student	PUBG
F	Programmer	Github
M	Programmer	Whatsapp
F	Programmer	Github
M	Student	PUBG
M	Student	PUBG

Example 1

Gender	Occupation	Suggestion
F	Student	PUBG
F	Programmer	Github
M	Programmer	Whatsapp
F	Programmer	Github
M	Student	PUBG
M	Student	PUBG

```
If occupation==student
    print(PUBG)
Else
    If gender==female
        print(Github)
    Else
        print(Whatsapp)
```

Where is the Tree?



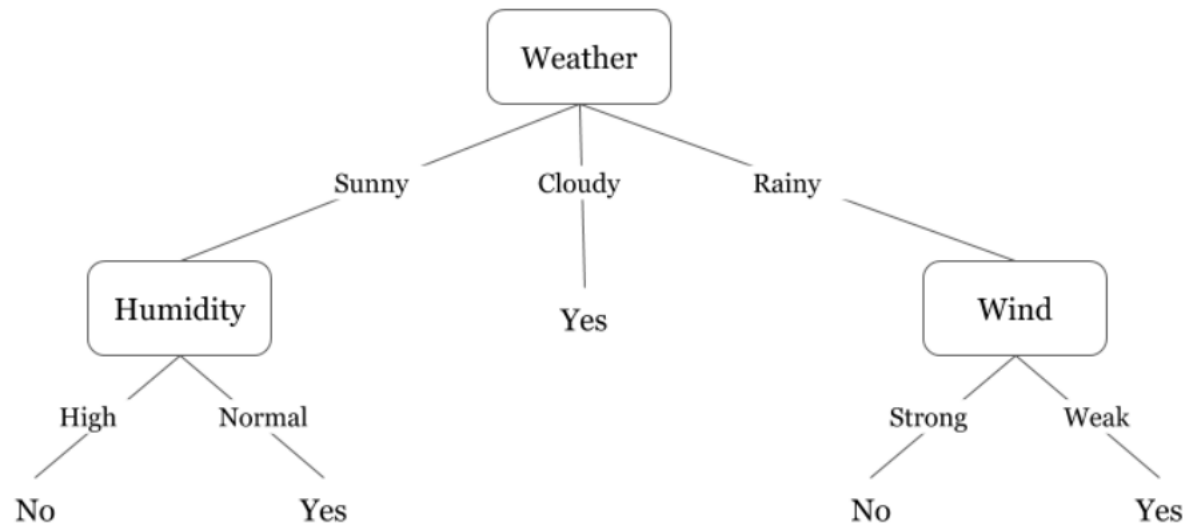
```
If occupation==student
    print(PUBG)
Else
    If gender==female
        print(Github)
    Else
        print(Whatsapp)
```



Example 2

Day	Weather	Temperature	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Cloudy	Hot	High	Weak	Yes
3	Sunny	Mild	Normal	Strong	Yes
4	Cloudy	Mild	High	Strong	Yes
5	Rainy	Mild	High	Strong	No
6	Rainy	Cool	Normal	Strong	No
7	Rainy	Mild	High	Weak	Yes
8	Sunny	Hot	High	Strong	No
9	Cloudy	Hot	Normal	Weak	Yes
10	Rainy	Mild	High	Strong	No

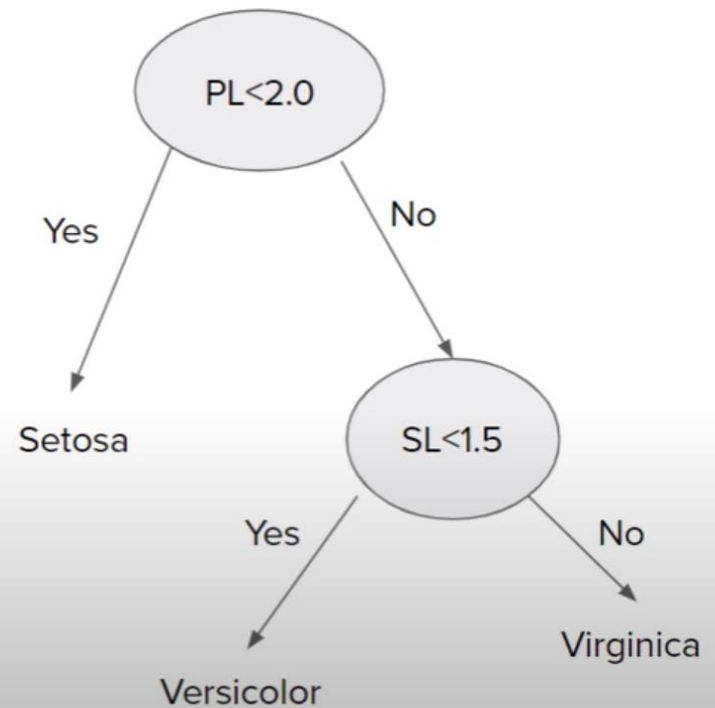
- In the below diagram the tree will first ask what is the weather? Is it sunny, cloudy, or rainy?
- If not cloudy then it will go to the next feature which is humidity and wind.
- It will again check if there is a strong wind or weak, if it's rainy and weak wind then the person may go and play.



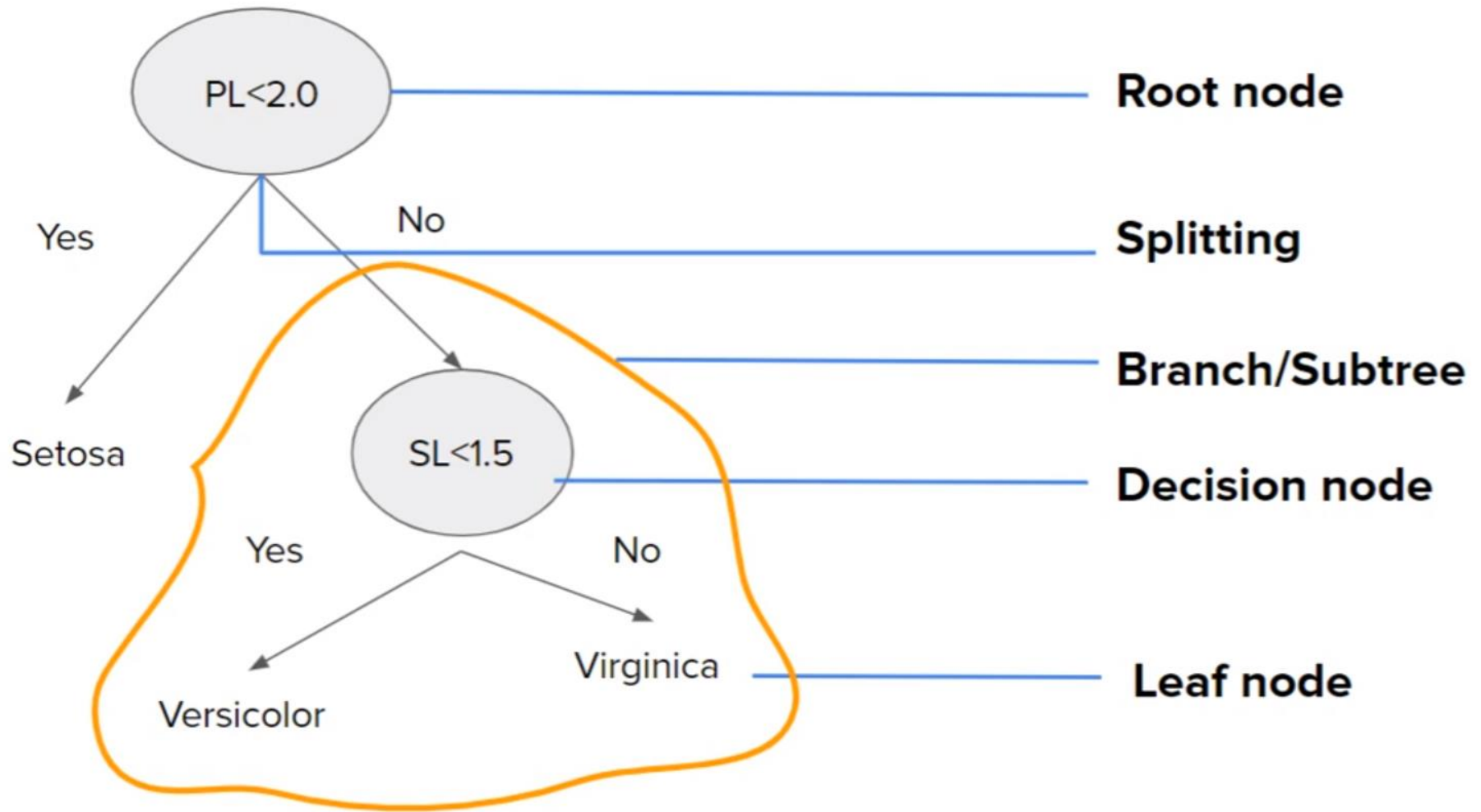
Example 3

What if we have numerical data?

Petal Length	Sepal Length	Type
1.34	0.34	Setosa
3.45	1.45	Versicolor
1.69	0.98	Setosa
2.56	1.79	Virginica
3.00	1.13	Versicolor
1.3	0.88	Setosa



Terminology



- A decision tree is a flowchart-like a tree structure where an internal node represents feature (or attribute), the branch represents a decision rule, and each leaf node represents the outcome.
- Decision trees are nothing but a bunch of if-else statements in layman terms. It checks if the condition is true and if it is then it goes to the next node attached to that decision.
- **Root Nodes** – It is the node present at the beginning of a decision tree from this node the population starts dividing according to various features.
- **Decision Nodes** – the nodes we get after splitting the root nodes are called Decision Node
- **Leaf Nodes** – the nodes where further splitting is not possible are called leaf nodes or terminal nodes
- **Sub-tree** – just like a small portion of a graph is called sub-graph similarly a sub-section of this decision tree is called sub-tree.

Some unanswered questions

How to decide which column should be considered as root node?

How to select subsequent decision nodes?

How to decide splitting criteria in case of numerical columns?

ID3 (Iterative Dichotomiser 3) Algorithm

- **Entropy**
- Entropy is nothing but the uncertainty in our dataset or measure of disorder.
- More knowledge less Entropy or More uncertainty higher the Entropy.
- **How to calculate Entropy?**

$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

- **Ex:** For two class problem:

$$E(S) = -p_{(+)} \log p_{(+)} - p_{(-)} \log p_{(-)}$$

- Here p_{+} is the probability of positive class
- p_{-} is the probability of negative class
- S is the subset of the training example

Example: 01

Salary	Age	Purchase
20000	21	Yes
10000	45	No
60000	27	Yes
15000	31	No
12000	18	No

$$H(d) = -P_y \log_2(P_y) - P_n \log_2(P_n)$$

$$H(d) = -2/5 \log_2(2/5) - 3/5 \log_2(3/5)$$

$$H(d) = 0.97$$

Salary	Age	Purchase
34000	31	No
15000	25	No
69000	57	Yes
25000	21	No
32000	28	No

$$H(d) = -P_y \log_2(P_y) - P_n \log_2(P_n)$$

$$H(d) = -1/5 \log_2(1/5) - 4/5 \log_2(4/5)$$

$$H(d) = 0.72$$

Example: 02

Salary	Age	Purchase
20000	21	No
10000	45	No
60000	27	No
15000	31	No
12000	18	No

$$H(d) = -P_y \log_2(P_y) - P_n \log_2(P_n)$$

$$H(d) = -0/5 \log_2(0/5) - 5/5 \log_2(5/5)$$

$$H(d) = 0$$

Entropy for 3 Class Problem

Salary	Age	Purchase
20000	21	Yes
10000	45	No
60000	27	Yes
15000	31	No
30000	30	Maybe
12000	18	No
40000	40	Maybe
20000	20	Maybe

$$H(d) = -P_y \log_2(P_y) - P_n \log_2(P_n) - P_m \log_2(P_m)$$

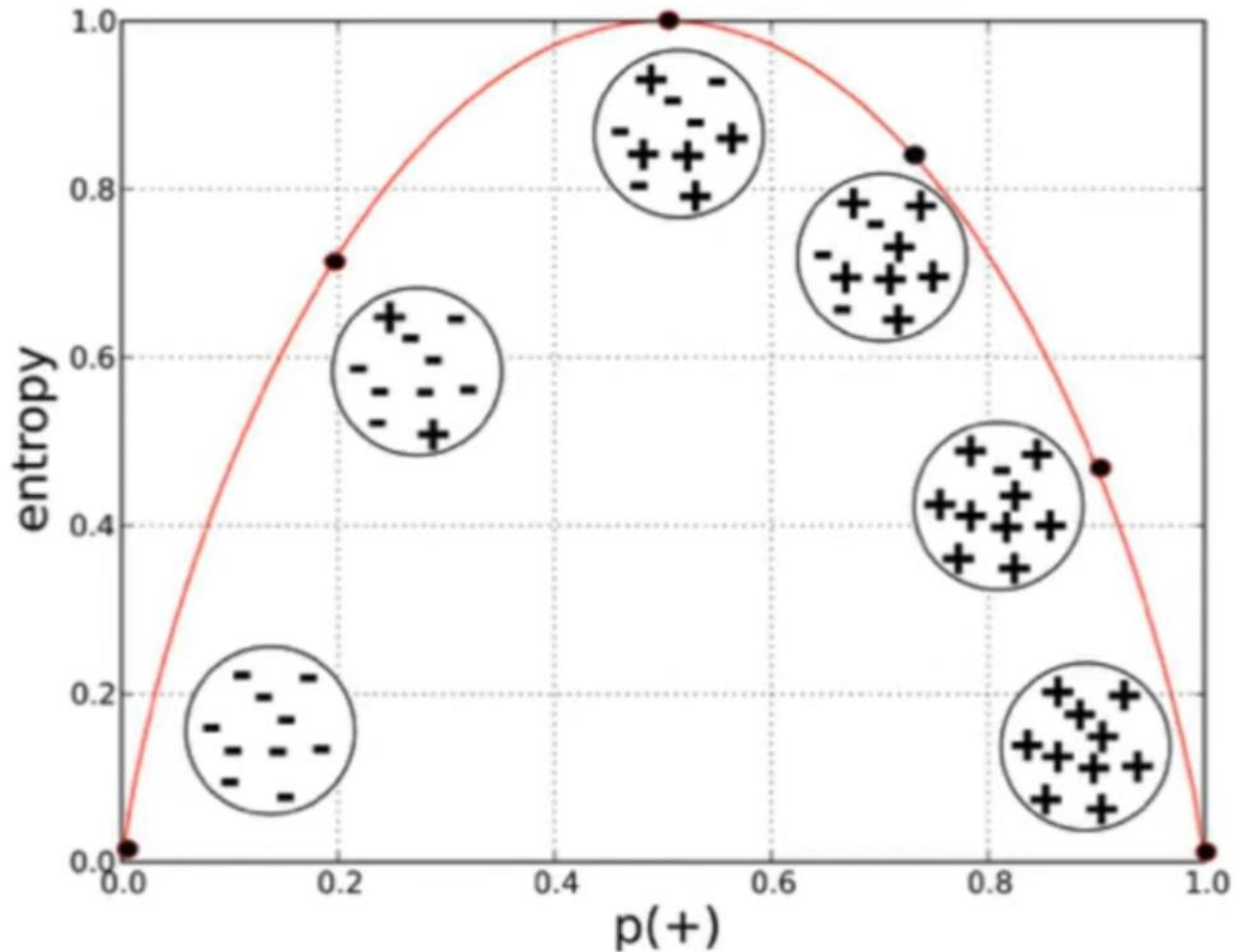
$$H(d) = -2/8 \log_2(2/8) - 3/8 \log_2(3/8) - 3/8 \log_2(3/8)$$

$$H(d) = 1.56$$

Observations

- More the uncertainty more is entropy
- For a 2 class problem the min entropy is 0 and the max is 1
- For more than 2 classes the min entropy is 0 but the max can be greater than 1

Entropy vs. Probability



Information Gain

- Information gain measures the reduction of uncertainty given some feature i.e., the information gain is based on the decrease in entropy after a dataset is split on an feature.
- It is also a deciding factor for which feature/attribute should be selected as a decision node or root node.

$$\text{Information Gain} = E(\text{Parent}) - \{\text{Weighted Average}\} * E\{\text{Children}\}$$

Example: 01

Outlook	Temperature	Humidity	Windy	PlayTennis
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

Step 1:

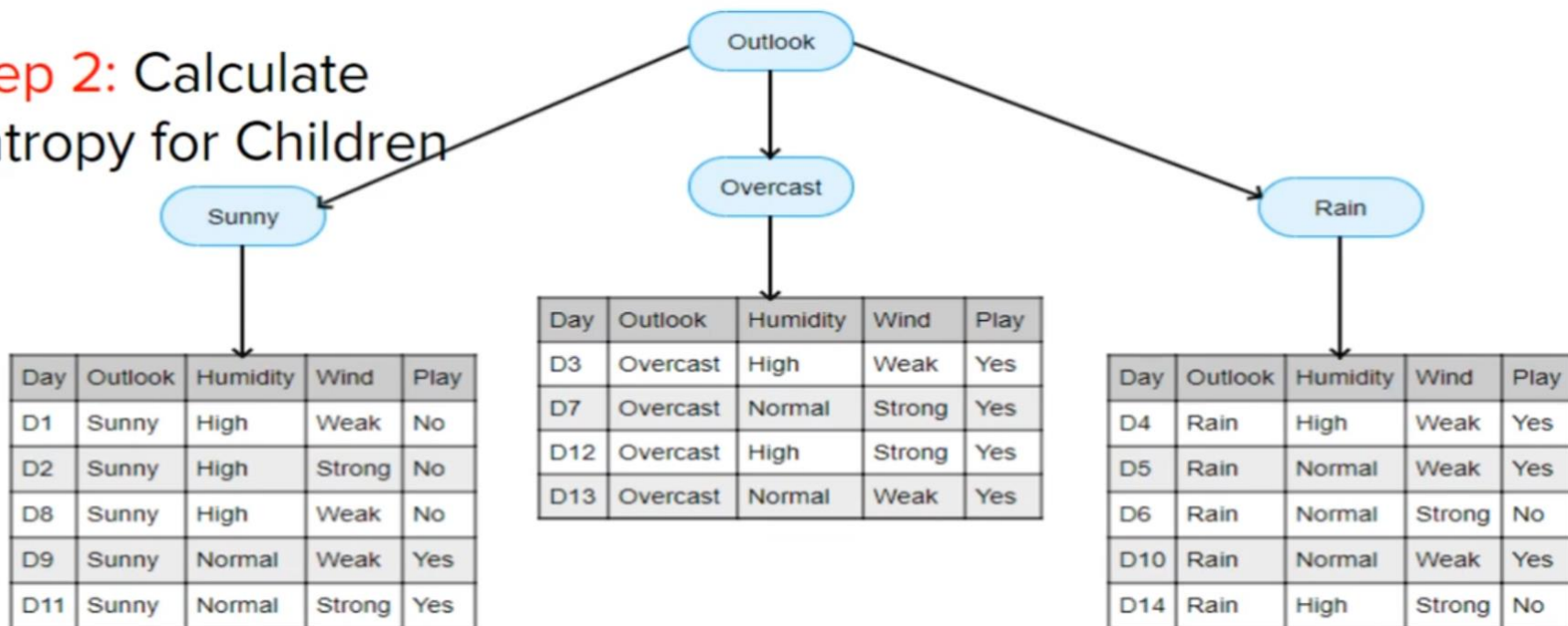
Entropy of Parent

$$E(P) = -p_y \log_2(p_y) - p_n \log_2(p_n)$$

$$= 9/14 \log_2(9/14) - 5/14 \log_2(5/14)$$

$$E(P) = \mathbf{0.94}$$

Step 2: Calculate Entropy for Children



$$E(S) = -2/5 \log(2/5) - 3/5 \log(3/5)$$

$$E(S) = 0.97$$

$$E(O) = -5/5 \log(5/5) - 0/5 \log(0/5)$$

$$E(O) = 0$$

$$E(R) = -3/5 \log(3/5) - 2/5 \log(2/5)$$

$$E(R) = 0.97$$

Step 3 : Calculate weighted Entropy of Children

$$\text{Weighted Entropy} = 5/14 * 0.97 + 4/14 * 0 + 5/14 * 0.97$$

$$\text{W.E(Children)} = \mathbf{0.69}$$

P(Overcast) is a leaf node as it's entropy is 0

Step 4 : Calculate Information Gain

Information Gain = $E(\text{Parent}) - \{\text{Weighted Average}\} * E(\text{Children})$

$$IG = 0.97 - 0.69 = 0.28$$

So the information gain(or the decrease in entropy/impurity) when you split this data on the basis of **Outlook** condition/column is **0.28**

Information Gain | Decision Tree Part 6

Step 5 : Calculate Information Gain for all the columns

Whichever column has the highest Information Gain(maximum decrease in entropy) the algorithm will select that column to split the data.

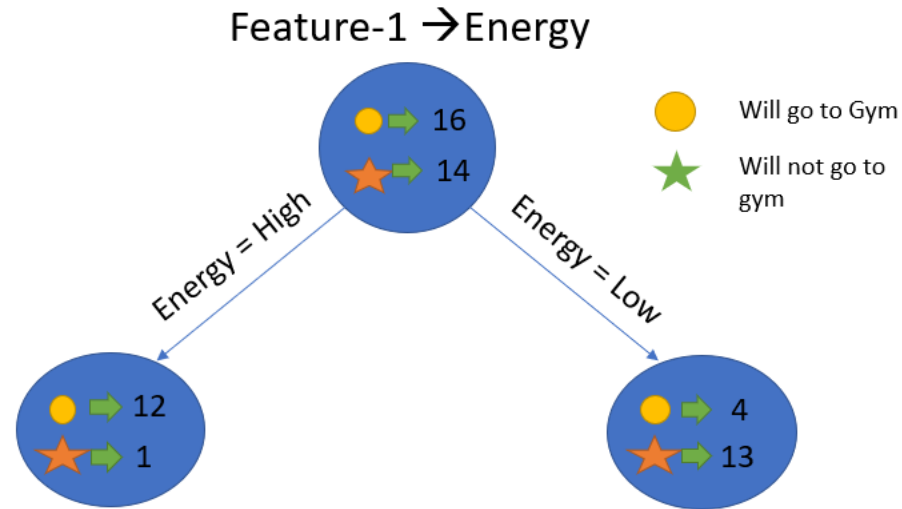
Step 6 : Find Information Gain recursively

Decision tree then applies a recursive greedy search algorithm in top bottom fashion to find Information Gain at every level of the tree.

Once a leaf node is reached (Entropy = 0), no more splitting is done.

Example: 02

- Suppose the entire population has a total of 30 instances. The dataset is to predict whether the person will go to the gym or not. Let's say 16 people go to the gym and 14 people don't.
- Now we have two features to predict whether he/she will go to the gym or not.
- Feature 1 is “**Energy**” which takes two values “*high*” and “*low*”.
- Feature 2 is “**Motivation**” which takes 3 values “*No motivation*”, “*Neutral*” and “*Highly motivated*”.
- Let's see how our decision tree will be made using these 2 features. We'll use information gain to decide which feature should be the root node and which feature should be placed after the split.



Let's calculate the entropy:

$$E(\text{Parent}) = -\left(\frac{16}{30}\right)\log_2\left(\frac{16}{30}\right) - \left(\frac{14}{30}\right)\log_2\left(\frac{14}{30}\right) \approx 0.99$$

$$E(\text{Parent}|\text{Energy} = \text{"high"}) = -\left(\frac{12}{13}\right)\log_2\left(\frac{12}{13}\right) - \left(\frac{1}{13}\right)\log_2\left(\frac{1}{13}\right) \approx 0.39$$

$$E(\text{Parent}|\text{Energy} = \text{"low"}) = -\left(\frac{4}{17}\right)\log_2\left(\frac{4}{17}\right) - \left(\frac{13}{17}\right)\log_2\left(\frac{13}{17}\right) \approx 0.79$$

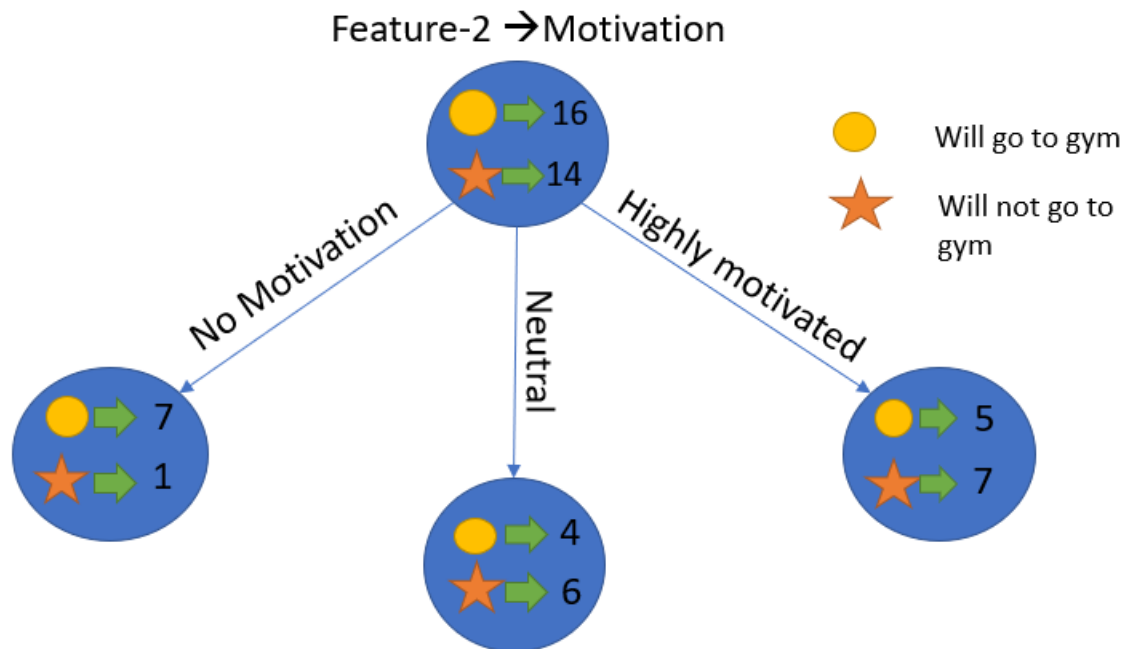
To see the weighted average of entropy of each node we will do as follows:

$$E(\text{Parent}|\text{Energy}) = \frac{13}{30} * 0.39 + \frac{17}{30} * 0.79 = 0.62$$

- Now we have the value of $E(\text{Parent})$ and $E(\text{Parent} | \text{Energy})$, information gain will be:

$$\begin{aligned}\text{Information Gain} &= E(\text{parent}) - E(\text{parent} | \text{energy}) \\ &= 0.99 - 0.62 \\ &= 0.37\end{aligned}$$

- Our parent entropy was near 0.99 and after looking at this value of information gain, we can say that the entropy of the dataset will decrease by 0.37 if we make “Energy” as our root node.
- Similarly, we will do this with the other feature “Motivation” and calculate its information gain.



Let's calculate the entropy here:

$$E(\text{Parent}) = 0.99$$

$$E(\text{Parent} | \text{Motivation} = \text{"No motivation"}) = -\left(\frac{7}{8}\right)\log_2\left(\frac{7}{8}\right) - \frac{1}{8}\log_2\left(\frac{1}{8}\right) = 0.54$$

$$E(\text{Parent} | \text{Motivation} = \text{"Neutral"}) = -\left(\frac{4}{10}\right)\log_2\left(\frac{4}{10}\right) - \left(\frac{6}{10}\right)\log_2\left(\frac{6}{10}\right) = 0.97$$

$$E(\text{Parent} | \text{Motivation} = \text{"Highly motivated"}) = -\left(\frac{5}{12}\right)\log_2\left(\frac{5}{12}\right) - \left(\frac{7}{12}\right)\log_2\left(\frac{7}{12}\right) = 0.98$$

- To see the weighted average of entropy of each node we will do as follows:

$$E(\text{Parent}|\text{Motivation}) = \frac{8}{30} * 0.54 + \frac{10}{30} * 0.97 + \frac{12}{30} * 0.98 = 0.86$$

- Now we have the value of $E(\text{Parent})$ and $E(\text{Parent}|\text{Motivation})$, information gain will be:

$$\begin{aligned}\text{Information Gain} &= E(\text{Parent}) - E(\text{Parent}|\text{Motivation}) \\ &= 0.99 - 0.86 \\ &= 0.13\end{aligned}$$

- We now see that the “Energy” feature gives more reduction which is 0.37 than the “Motivation” feature. Hence we will select the feature which has the highest information gain and then split the node based on that feature.

Example: 03

Age	Competition	Type	Profit
old	yes	software	down
old	no	software	down
old	no	hardware	down
mid	yes	software	down
mid	yes	hardware	down
mid	no	hardware	up
mid	no	software	up
new	yes	software	up
new	no	hardware	up
new	no	software	up

- This dataset has 10 instances and four numbers of attributes. Here, first three attributes are predictor and the last attribute is the target attribute.

- **Solution:**

- $E(\text{Profit}) = 1$

- $E(\text{Age}=\text{old}) = 0$

- $E(\text{Age}=\text{mid}) = 1$

- $E(\text{Age}=\text{new}) = 0$

- $E(\text{Age}) = 0.4$

- $IG = 1 - 0.4$
 $= 0.6$

Age	Down	Up
Old	3	0
Mid	2	2
New	0	3

- Similarly,

- $E(\text{Competition}=\text{yes}) = 0.81$

- $E(\text{Competition}=\text{no}) = 0.91$

- $E(\text{Competition}) = 0.88$

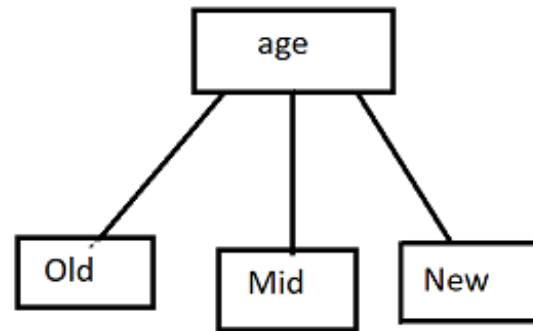
- $IG(\text{Competition}) = 0.12$

- $E(\text{Type}=\text{software}) = 1$
- $E(\text{Type}=\text{hardware}) = 1$
- $E(\text{Type}) = 1$
- $IG(\text{Type}) = 0$

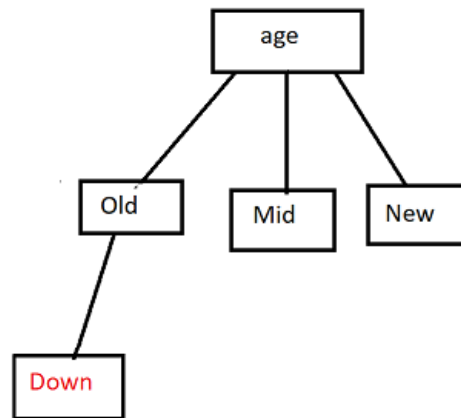
- **So, Gain of (age, competition, type)**
- Information Gain of age = 0.6
- Information Gain of competition = 0.12
- Information Gain of type = 0

- As Age has a max gain as compared to others so age becomes a root node.

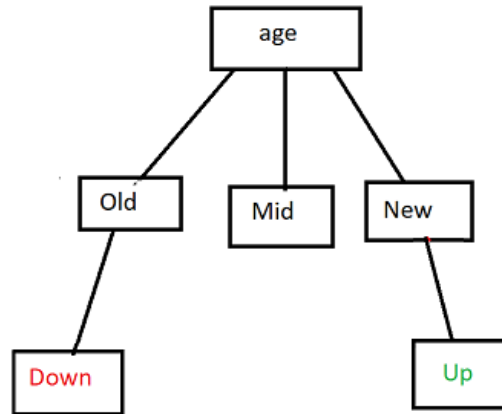
- So age has three attributes which are Old, mid and new.



- For old age target attribute, all goes down. So make a leaf node for old which is down.

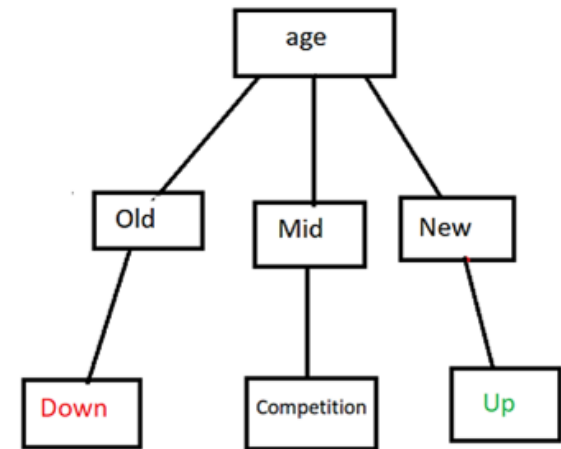


- For new age target attribute all go up. So make a leaf node for new age.

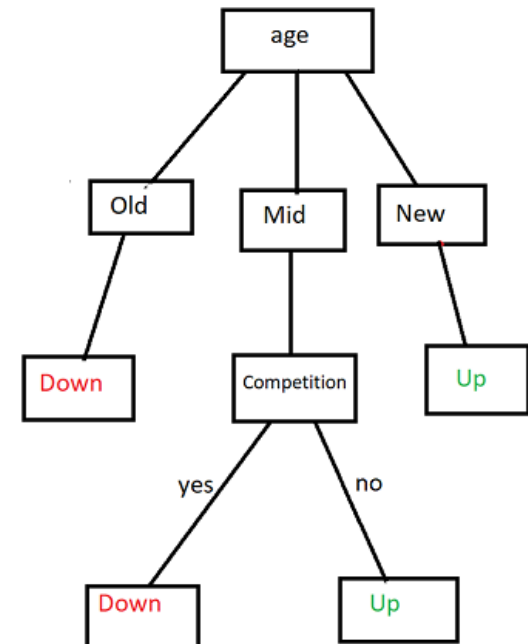


- Now what is for mid? Because mid age has 2 cases.
- So who becomes the child node of mid.
- We take that attribute for which information gain is higher among Competition and Type.
- Information Gain of competition = 0.12
- Information Gain of type = 0

- So, Information Gain of Competition is Greater than Type, so Competition becomes the child node of mid.



- Now, make a leaf node for competition. If the competition is yes then the profit will down and if competition no then profits up.



Gini Impurity (Gini Index)

- Gini impurity is a measure used to quantify a dataset's impurity level or disorder.
- It is one of the methods used in decision tree algorithms to decide the optimal split from a root node, and subsequent splits.
- It ranges from 0 to 0.5, where 0 indicates a perfectly pure node (all instances belong to the same class), and 0.5 signifies maximum impurity (an equal distribution of classes).
- Gini Impurity is calculated using the formula: $GI = 1 - \sum_{i=1}^n (p_i)^2$
- **Ex: For 2-class dataset** $GI = 1 - [(P_{(+)})^2 + (P_{(-)})^2]$

Example

Salary	Age	Purchase
20000	21	Yes
10000	45	No
60000	27	Yes
15000	31	No
12000	18	No

$$G = 1 - (P_y^2 + P_n^2)$$

$$G = 1 - (4/25 + 9/25)$$

$$G = 0.48$$

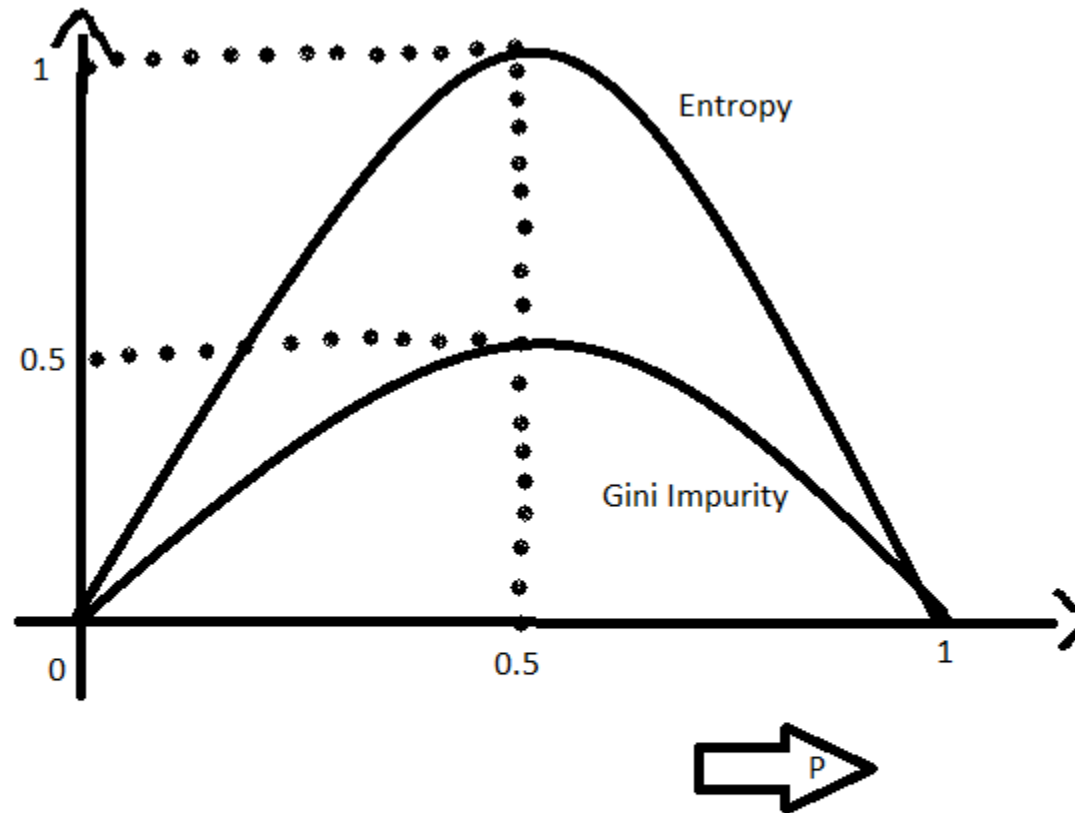
Salary	Age	Purchase
34000	31	No
15000	25	No
69000	57	Yes
25000	21	No
32000	28	No

$$G = 1 - (P_y^2 + P_n^2)$$

$$G = 1 - (1/25 + 16/25)$$

$$G = 0.32$$

Entropy v/s Gini Impurity



$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

$$Gini(E) = 1 - \sum_{j=1}^c p_j^2$$

Gini Impurity

- Formula for the Gini index is

$$GI = 1 - \sum (P_i)^2 ,$$

where P_i is probability of class i .

- It is the probability of misclassifying a randomly chosen element in a set.
- For binary classification, the range of the Gini index is $[0, 0.5]$, where 0 indicates perfect purity and 0.5 indicates maximum impurity.
- Gini index is a linear measure.
- The computational complexity of the Gini index is less.
- It is less robust than entropy.

Entropy

- Formula for entropy is:

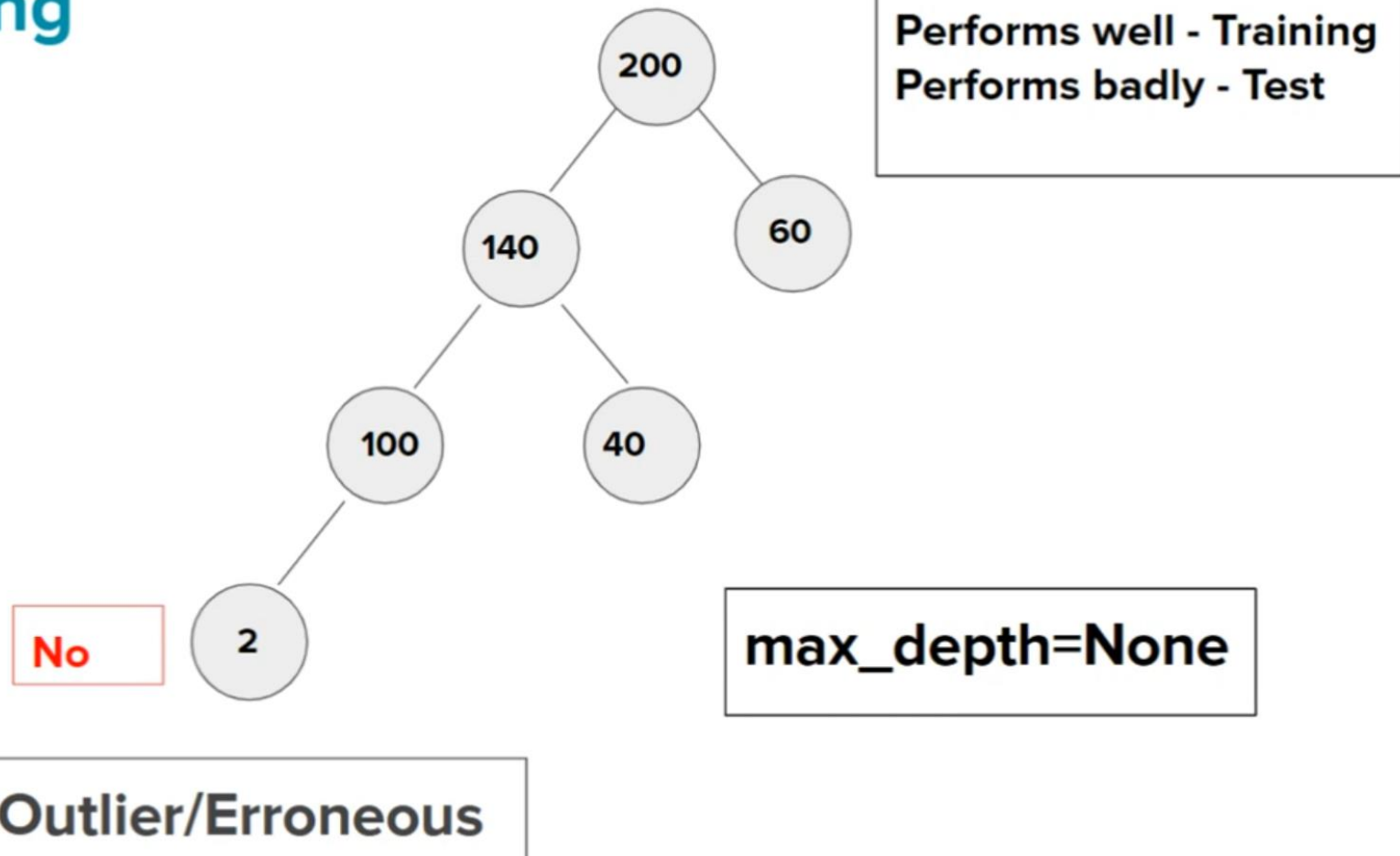
$$\text{Entropy} = -\sum (P_i) \log(P_i),$$

where p_i is probability of class i .

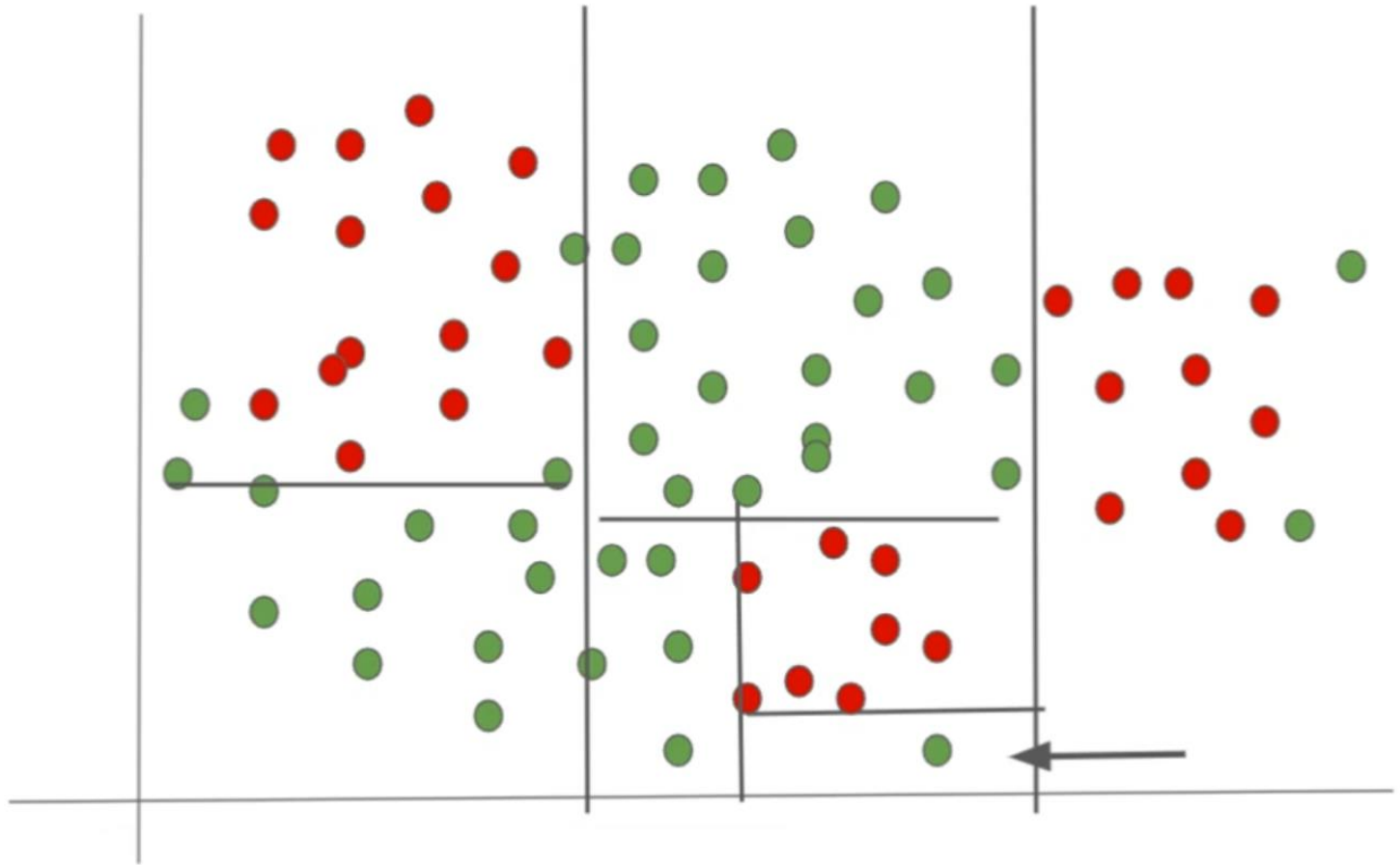
- Entropy measures the amount of uncertainty or randomness in a set.
- For binary classification, the range of the Entropy is $[0, 1]$, where 0 indicates perfect purity and 1 indicates maximum impurity.
- Entropy is a logarithmic measure.
- The computational complexity of Entropy is more.
- It is more robust than Gini index.

Overfitting and Underfitting in Decision Tree

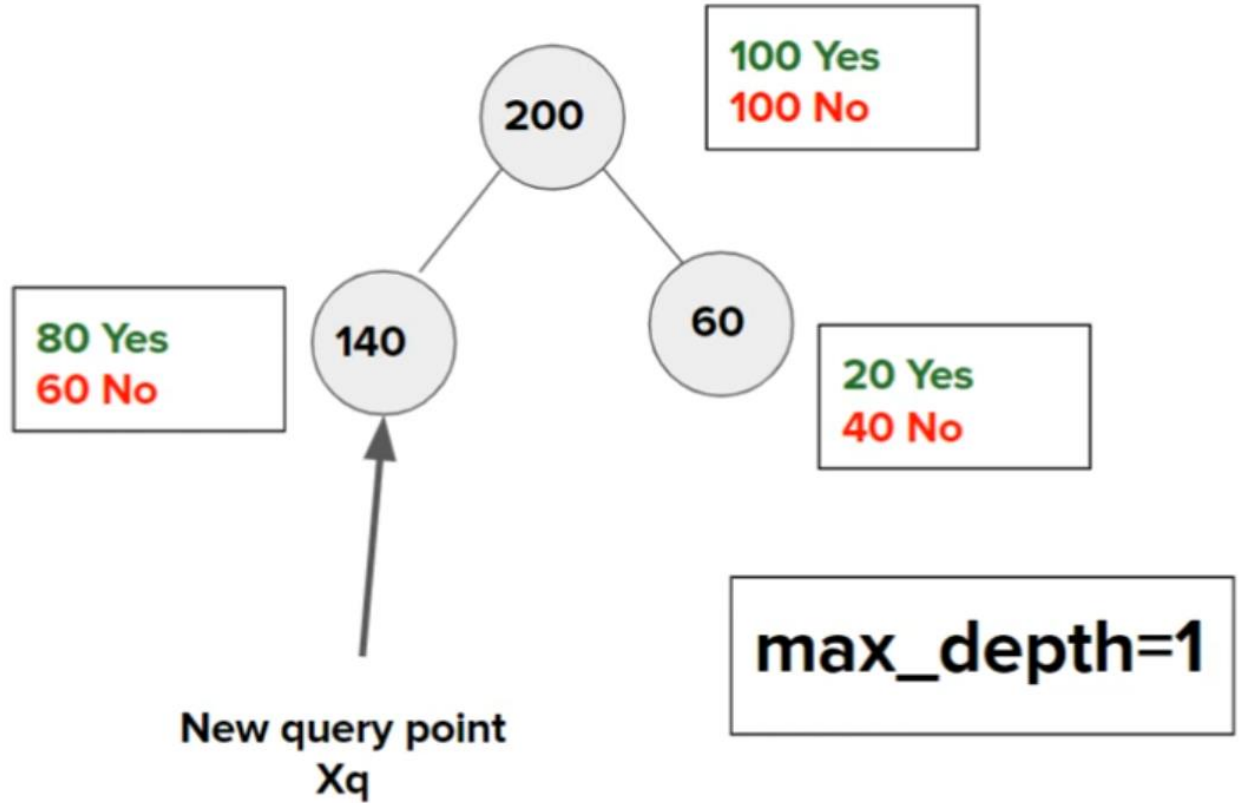
Overfitting



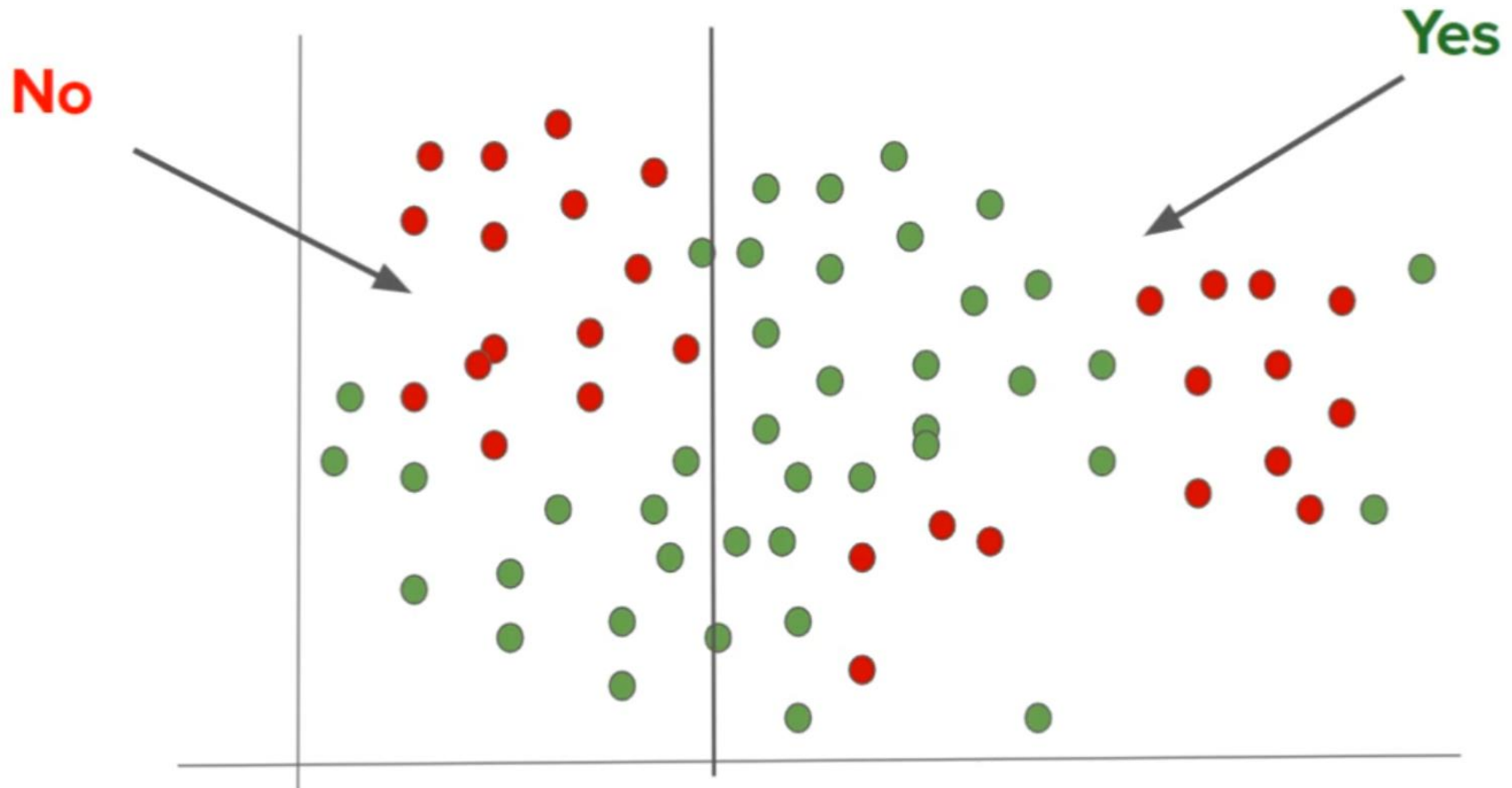
Geometric Intuition of Overfitting



Underfitting



Geometric Intuition of Underfitting



Pruning

- Pruning is a technique associated with decision trees.
- Pruning reduces the size of decision trees by removing parts of the tree.
- Effective pruning can reduce the overfitting.
- There are two types of pruning:
 - Pre-pruning and
 - Post-pruning.

Pre-pruning

- The pre-pruning technique is tuning the hyper-parameters prior to the training pipeline.
- It is also known as ‘early stopping’ which stops the growth of the decision tree - preventing it from reaching its full depth.
- During each stage of the splitting of the tree, the cross-validation error will be monitored.
- Cross-validation is the process of building a tree with most of the data and then using the remaining part of the data to test the accuracy of the decision tree.

- If the value of the error does not decrease anymore - then we stop the growth of the decision tree.
- The hyper-parameters that can be tuned for early stopping and preventing overfitting are: `max_depth`.
- However, one should be cautious as early stopping can also lead to underfitting.

Post-pruning

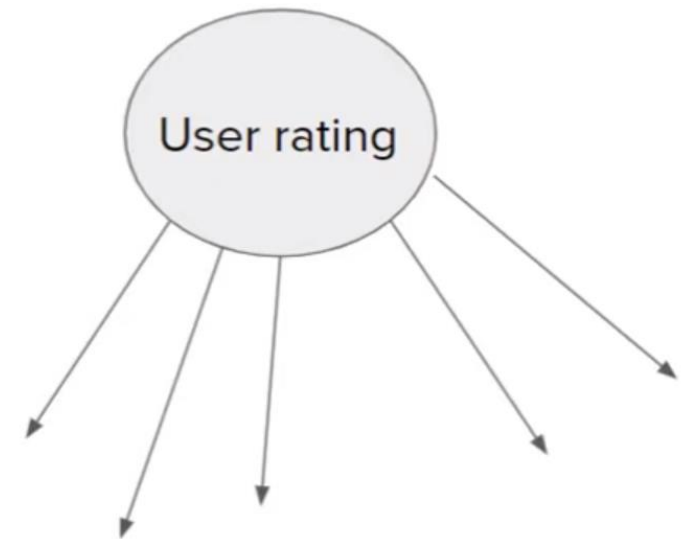
- Post-pruning does the opposite of pre-pruning and allows the Decision Tree model to grow to its full depth.
- Once the model grows to its full depth, tree branches are removed to prevent the model from overfitting.
- This is done by segregating the actual training set into two sets: training data set, D and validation data set, V .
- Prepare the decision tree using the segregated training data set, D . Then continue trimming the tree accordingly to optimize the accuracy of the validation data set, V .

Handling Numerical Values in DT

Example:

S No	User Rating	Downloaded
1	3.5	Yes
2	4.6	Yes
3	2.2	No
4	1.6	Yes
5	4.1	No
6	3.9	No
7	3.2	No
9	2.9	Yes
10	4.8	Yes
11	3.3	No
12	2.5	Yes
13	1.9	Yes

```
data['user_rating'].nunique()-1
```



We will then have n subtrees

Solution

Step 1:

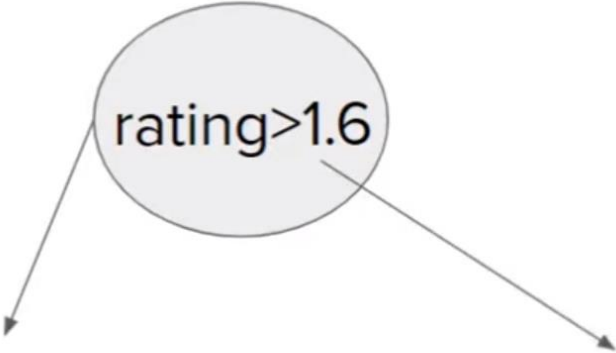
Sort the data on the basis of numerical column

S No	User Rating	Downloaded
1	1.6	Yes
2	1.9	Yes
3	2.2	No
4	2.5	Yes
5	2.9	Yes
6	3.2	No
7	3.3	No
9	3.5	Yes
10	3.9	No
11	4.1	No
12	4.6	Yes
13	4.8	Yes

Step 2:

Split the entire data on the basis of every value of user_rating

rating > 1.6



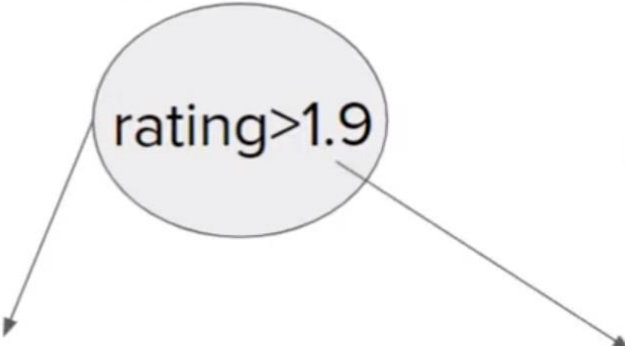
S No	User Rating	Downloaded
1	1.6	Yes

S No	User Rating	Downloaded
2	1.9	Yes
3	2.2	No
4	2.5	Yes
5	2.9	Yes
6	3.2	No
7	3.3	No
9	3.5	Yes
10	3.9	No
11	4.1	No
12	4.6	Yes
13	4.8	Yes

Step 2:

Split the entire data on the basis of every value of user_rating

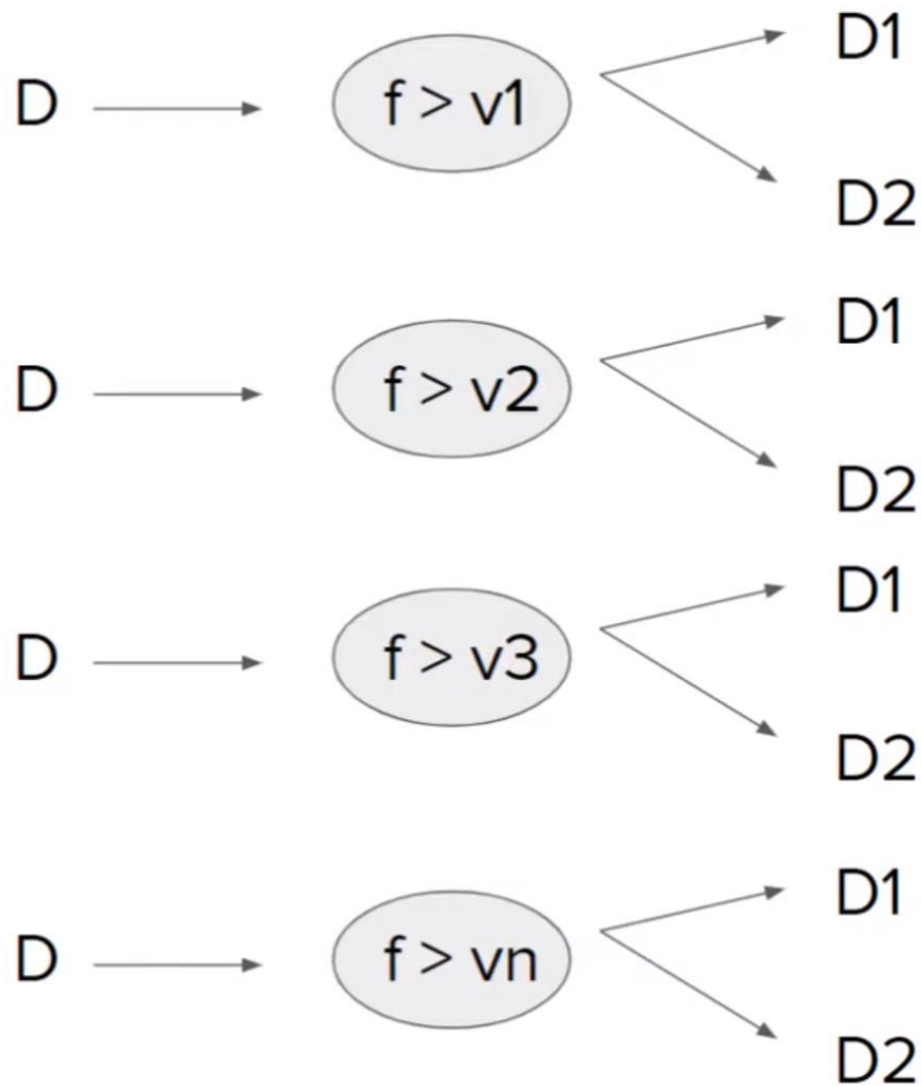
rating > 1.9



S No	User Rating	Downloaded
1	1.6	Yes
2	1.9	Yes

S No	User Rating	Downloaded
3	2.2	No
4	2.5	Yes
5	2.9	Yes
6	3.2	No
7	3.3	No
9	3.5	Yes
10	3.9	No
11	4.1	No
12	4.6	Yes
13	4.8	Yes

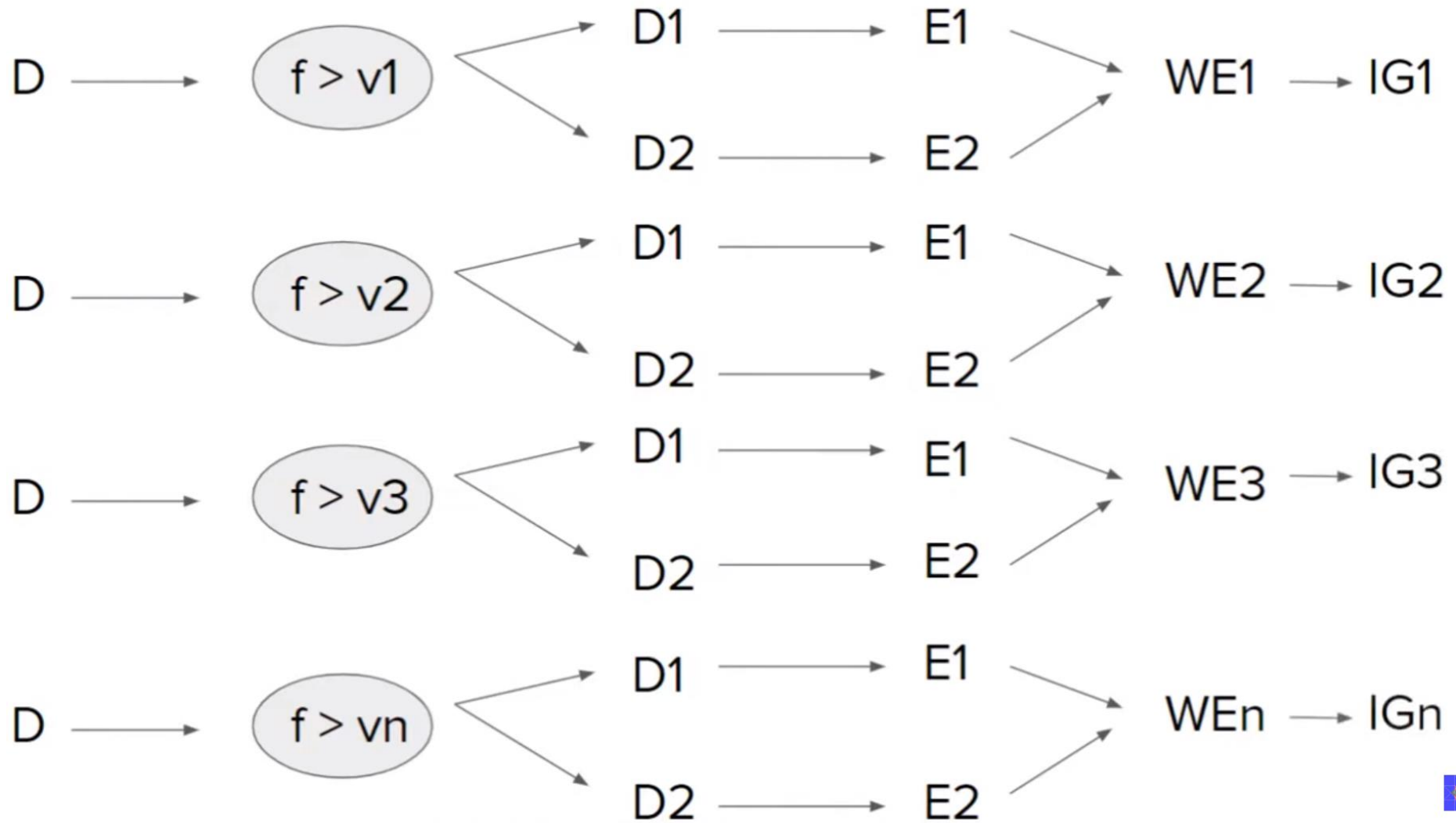
- Repeat this process for each value of User Rating.



- Where D is the Dataset
- f is the column User Rating
- $v1, v2, \dots, v_n$ are the values of various rows of User Rating

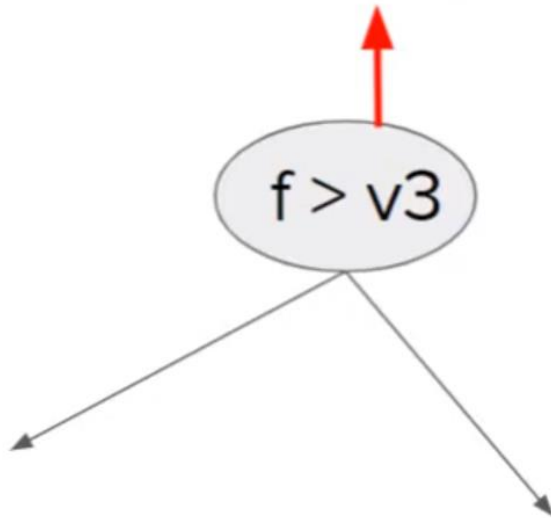
Step 3

Step 4



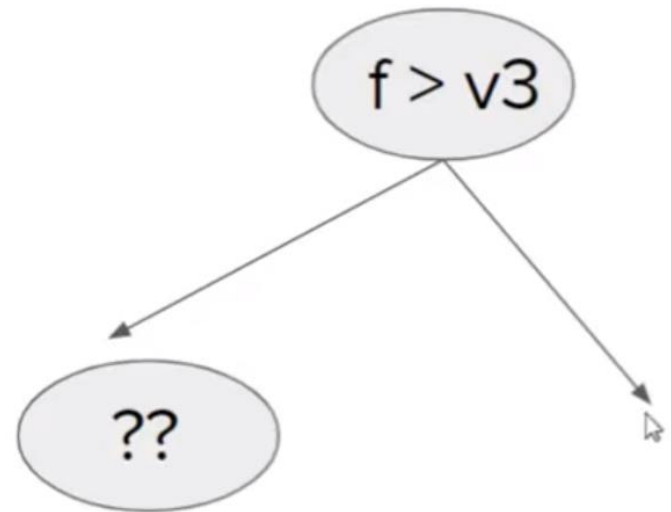
Step 5:

$\text{Max}\{ \text{IG1}, \text{IG2}, \text{IG3}....., \text{IGn} \}$



Step 6:

Do this recursively until you get all the leaf nodes



Advantages

Intuitive and easy to understand

Minimal data preparation is required

The cost of using the tree for inference is **logarithmic** in the number of data points used to train the tree

Disadvantages

Overfitting

Prone to errors for imbalanced datasets

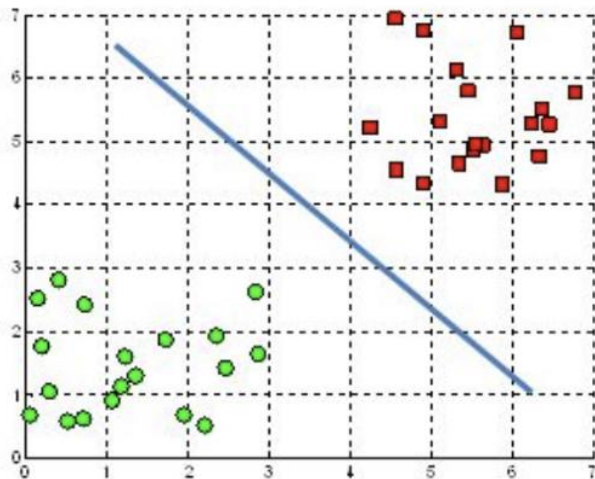
Support Vector Machine (SVM)

- A Support Vector Machine (SVM) is a supervised machine learning algorithm that can be employed for both classification and regression purposes.
- However, it is mostly used in classification problems.
- The goal of the SVM algorithm is to create the best decision boundary that can segregate n-dimensional space into classes so that we can easily put the test data point in the correct category.
- This best decision boundary is called a **hyperplane**.

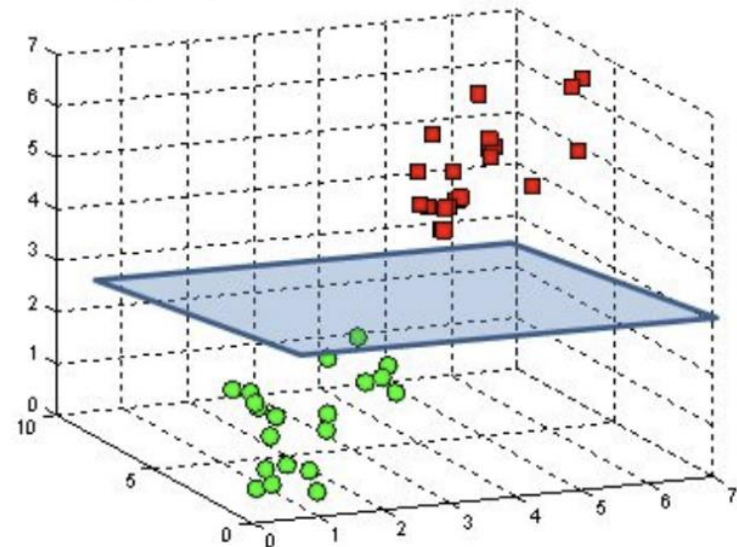
Hyperplane

- A hyperplane is a generalization of a plane.
 - in one dimension, a hyperplane is called a point.
 - in two dimensions, it is a line.
 - in three dimensions, it is a plane.
 - in more dimensions, we can call it an hyperplane.

A hyperplane in $\mathbb{R}^2$ is a line

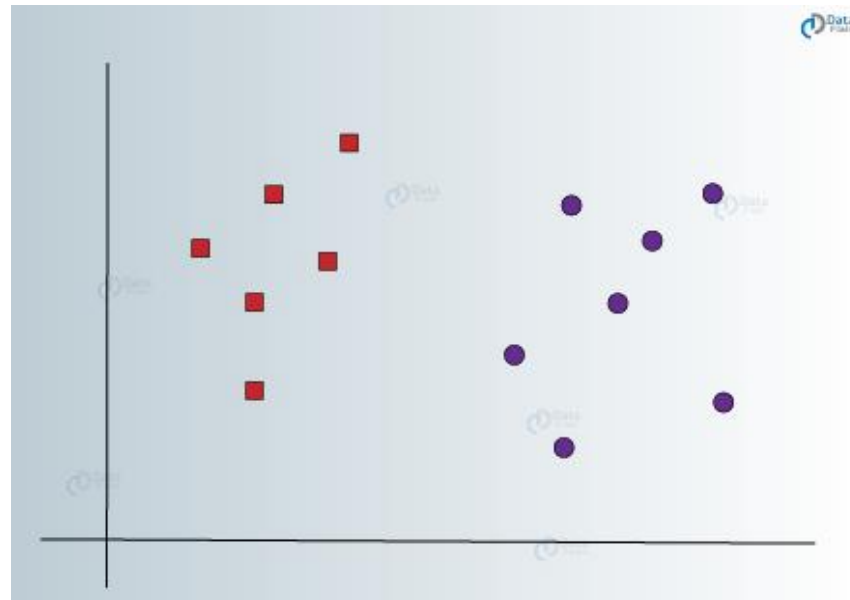


A hyperplane in $\mathbb{R}^3$ is a plane

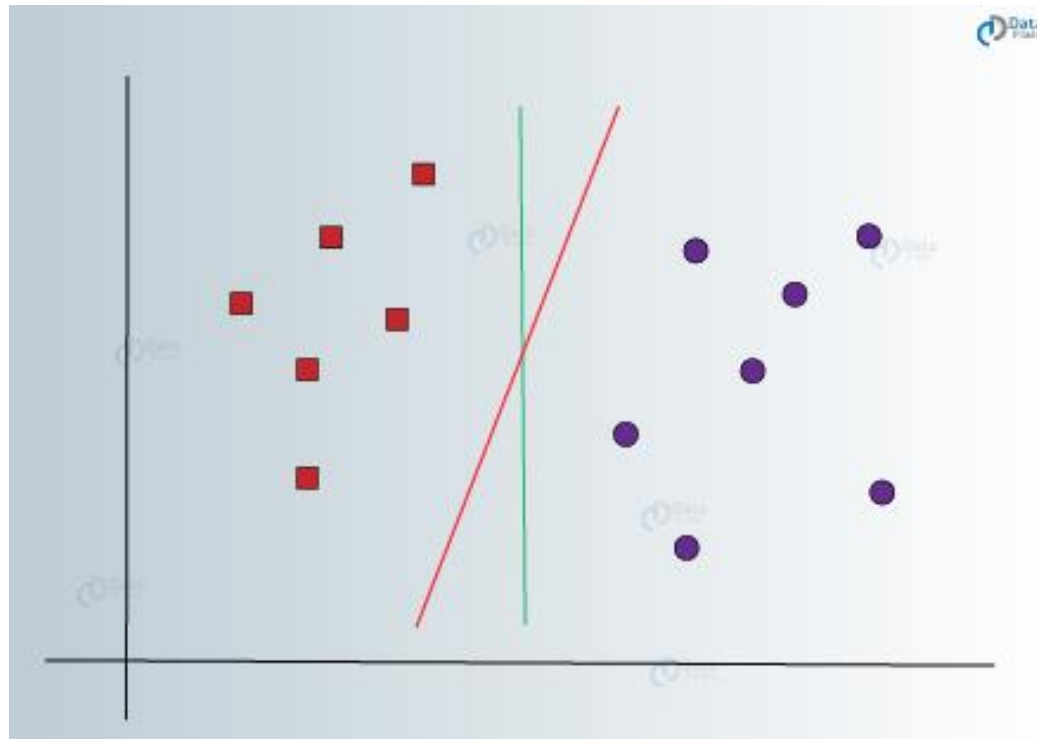


How does SVM work?

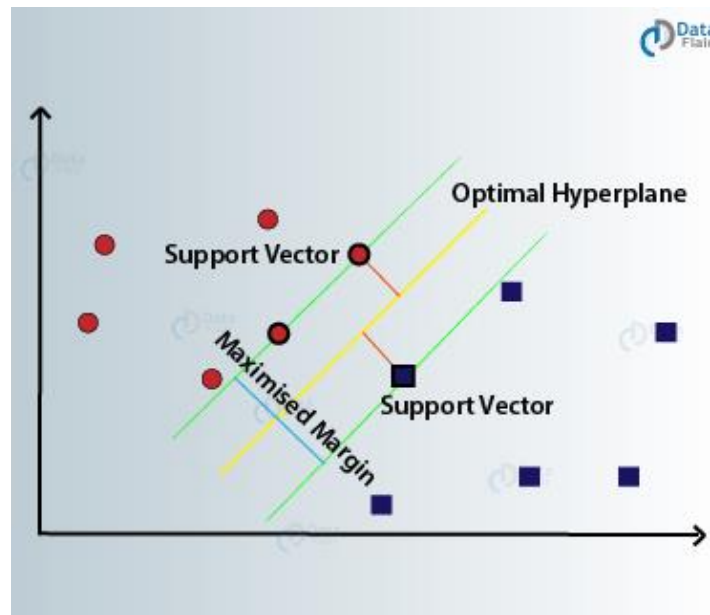
- The basic principle behind the working of SVM is to create a **hyperplane** that separates the dataset into classes.
- Suppose that for a given dataset, we have to classify red squares from blue circles. Our goal is to create a line that classifies the data into two classes, creating a distinction between red squares and blue circles.



- While one can hypothesize a clear line that separates the two classes, there can be many lines that can do this job.
- Therefore, there is not a single line that you can agree on which can perform this task.
- Let us visualize some of the lines that can differentiate between the two classes as follows —



- According to SVM, we have to find the points that lie closest to both the classes. These points are known as **support vectors** and hence, the algorithm is termed as Support Vector Machine
- The distance between the support vectors and the dividing line (decision boundary) is known as **margin**.
- The aim of an SVM algorithm is to **maximize this margin**.
- When the margin reaches its maximum, the hyperplane becomes the optimal one.



- The SVM model tries to maximize the distance between the two classes by creating a well-defined decision boundary.
- In the above case, hyperplane divided the data. While our data was in 2 dimensions, the hyperplane was of 1 dimension.
- For higher dimensions, say, an **n-dimensional Space**, we have an **n-1 dimensional hyperplane**.

What happens when there is no clear hyperplane?



- This type of dataset is called **non-linearly separable dataset**.
- In order to classify non-linearly separable dataset like the one above, we need to map the dataset into a higher dimension space using kernel functions.

Kernel Functions

- The kernel function maps the input data into a higher-dimensional space where the data becomes linearly separable.

- Common kernel functions for SVM

- linear

$$k(\mathbf{x}_1, \mathbf{x}_2) = \mathbf{x}_1 \cdot \mathbf{x}_2$$

- polynomial

$$k(\mathbf{x}_1, \mathbf{x}_2) = (\gamma \mathbf{x}_1 \cdot \mathbf{x}_2 + c)^d$$

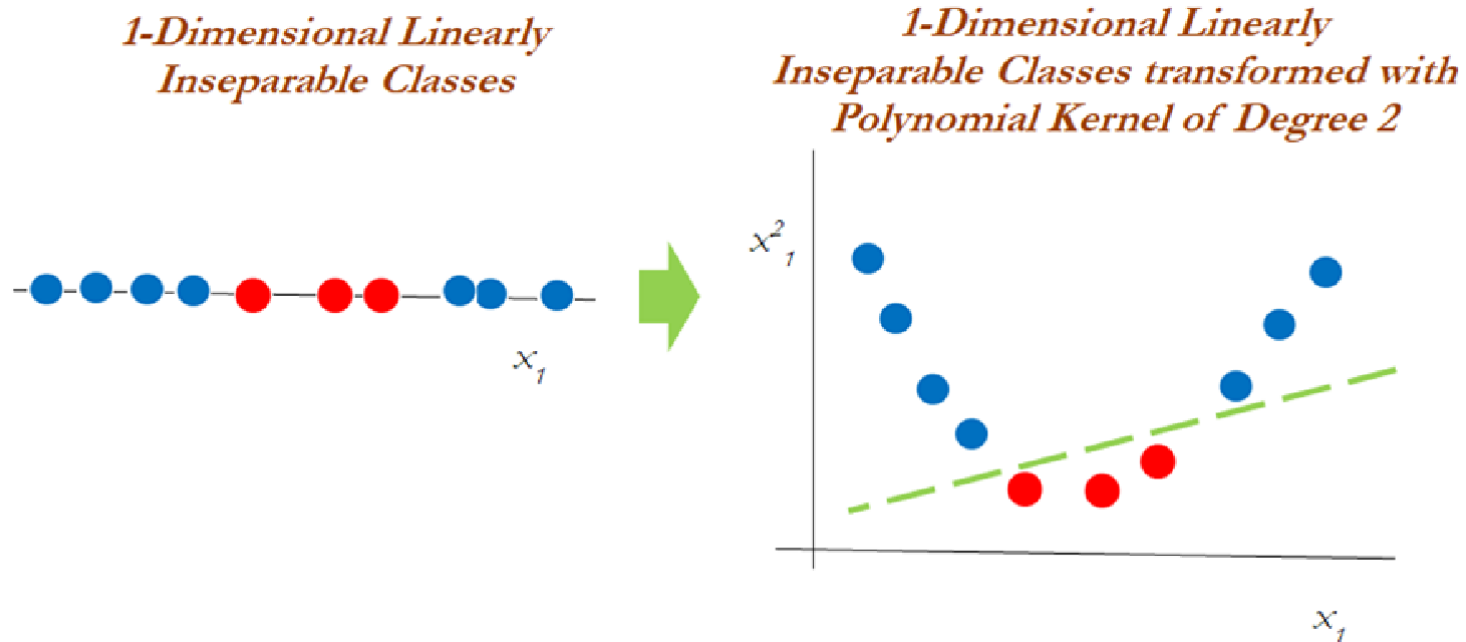
- Gaussian or radial basis

$$k(\mathbf{x}_1, \mathbf{x}_2) = \exp\left(-\gamma \|\mathbf{x}_1 - \mathbf{x}_2\|^2\right)$$

- sigmoid

$$k(\mathbf{x}_1, \mathbf{x}_2) = \tanh(\gamma \mathbf{x}_1 \cdot \mathbf{x}_2 + c)$$

- Another example of non-linearly separable dataset:
- **Example 2:**



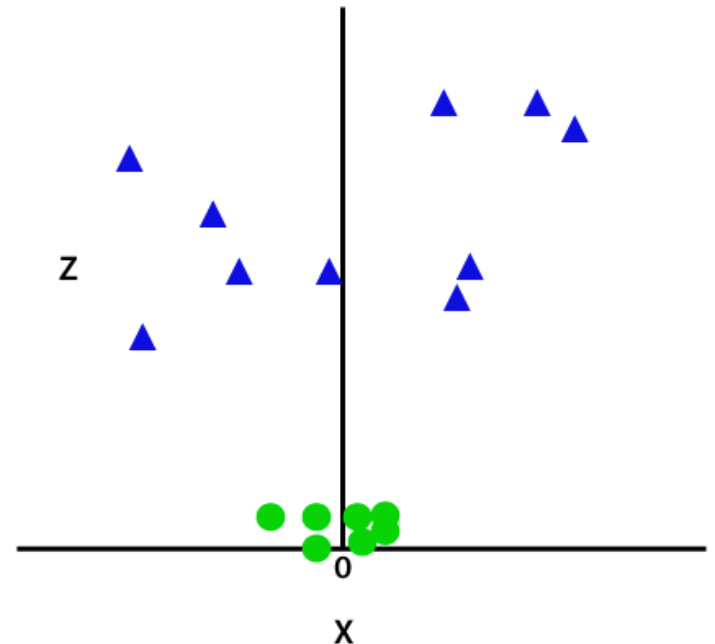
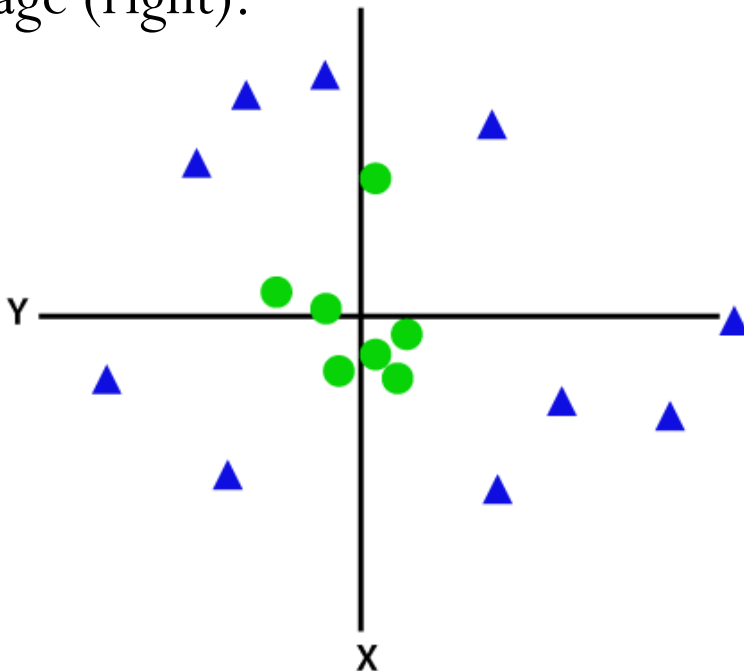
- For mathematical background of SVM:
- <https://www.robots.ox.ac.uk/~az/lectures/ml/lect2.pdf>

- **Example 3:**

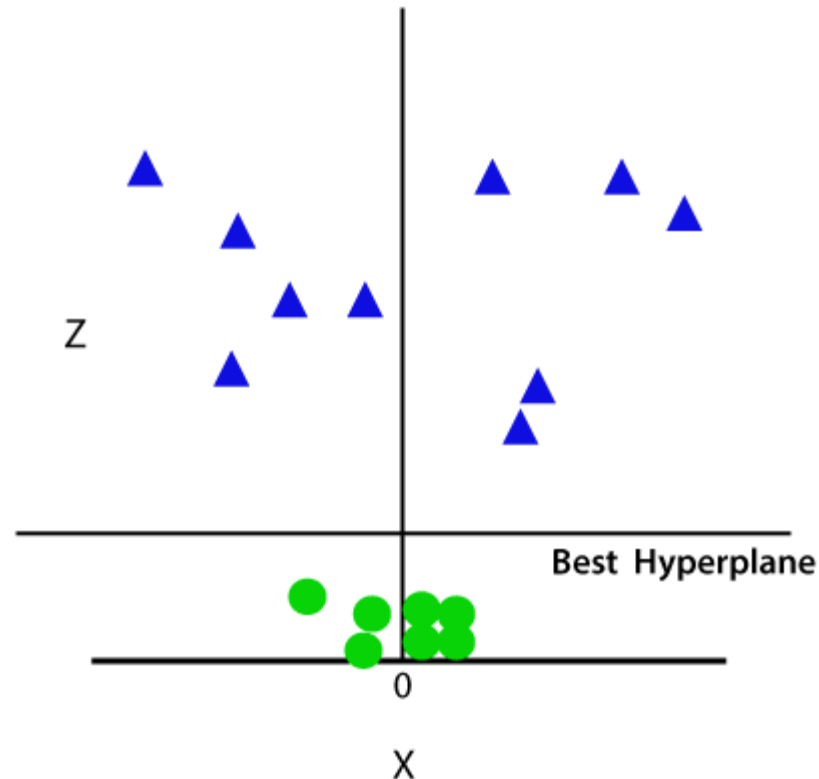
- To separate the data points as shown below (left), we need to add one more dimension. For linear data, we have used two dimensions x and y, so for non-linear data, we will add a third dimension z. It can be calculated as:

$$Z = x^2 + y^2$$

- By adding the third dimension, the sample space will become as below image (right):



- So now, SVM will divide the datasets into classes in the following way.



- Since we are in 3-d Space, hence it is looking like a plane parallel to the x-axis.

- SVM can be of two types:
- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

- **Advantages:**

- Ability to perform well with small datasets.
- Ability to handle high-dimensional data.
- They also have the ability to model non-linear decision boundaries, which can be very useful in many applications.

- **Disadvantages:**

- SVMs can be sensitive to the choice of kernel.
- It can be computationally expensive when the dataset is large.

Math behind SVM

Equation of hyperplane:

$$\vec{X} \cdot \vec{w} + b \geq 0$$

hence

$$y = \begin{cases} +1 & \text{if } \vec{X} \cdot \vec{w} + b \geq 0 \\ -1 & \text{if } \vec{X} \cdot \vec{w} + b < 0 \end{cases}$$

Hard Margin SVM:

$$\operatorname{argmax}(w^*, b^*) \frac{2}{\|w\|} \text{ such that } y_i(\vec{w} \cdot \vec{X} + b) \geq 1$$

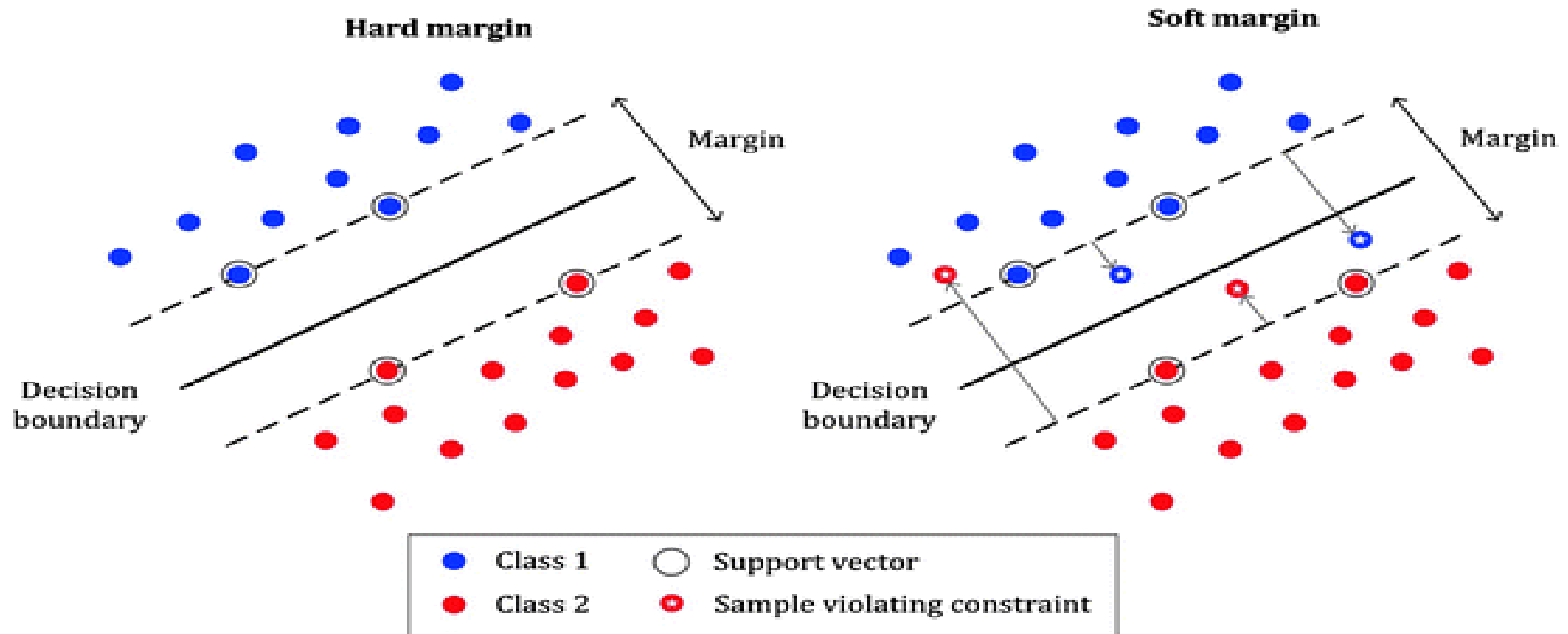
Margin:

$$d = \frac{2}{\|w\|}$$

Soft Margin SVM:

$$\operatorname{argmin}(w^*, b^*) \frac{\|w\|}{2} + c \sum_{i=1}^n \zeta_i$$

Hard Margin SVM Vs Soft Margin SVM



Numerical on SVM

Support Vector Machine - Linear Example Solved

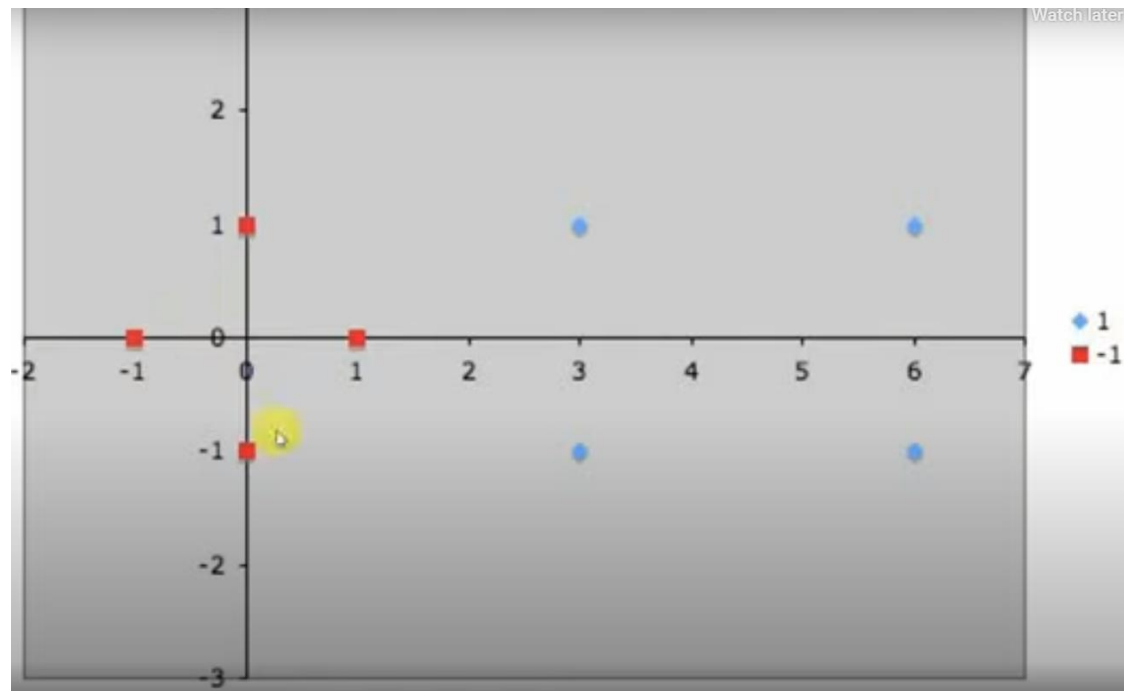
Suppose we are given the following positively labeled data points,

$$\left\{ \begin{pmatrix} 3 \\ 1 \end{pmatrix}, \begin{pmatrix} 3 \\ -1 \end{pmatrix}, \begin{pmatrix} 6 \\ 1 \end{pmatrix}, \begin{pmatrix} 6 \\ -1 \end{pmatrix} \right\}$$

and the following negatively labeled data points,

$$\left\{ \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ 0 \end{pmatrix} \right\}$$

Graphical representation of the given data

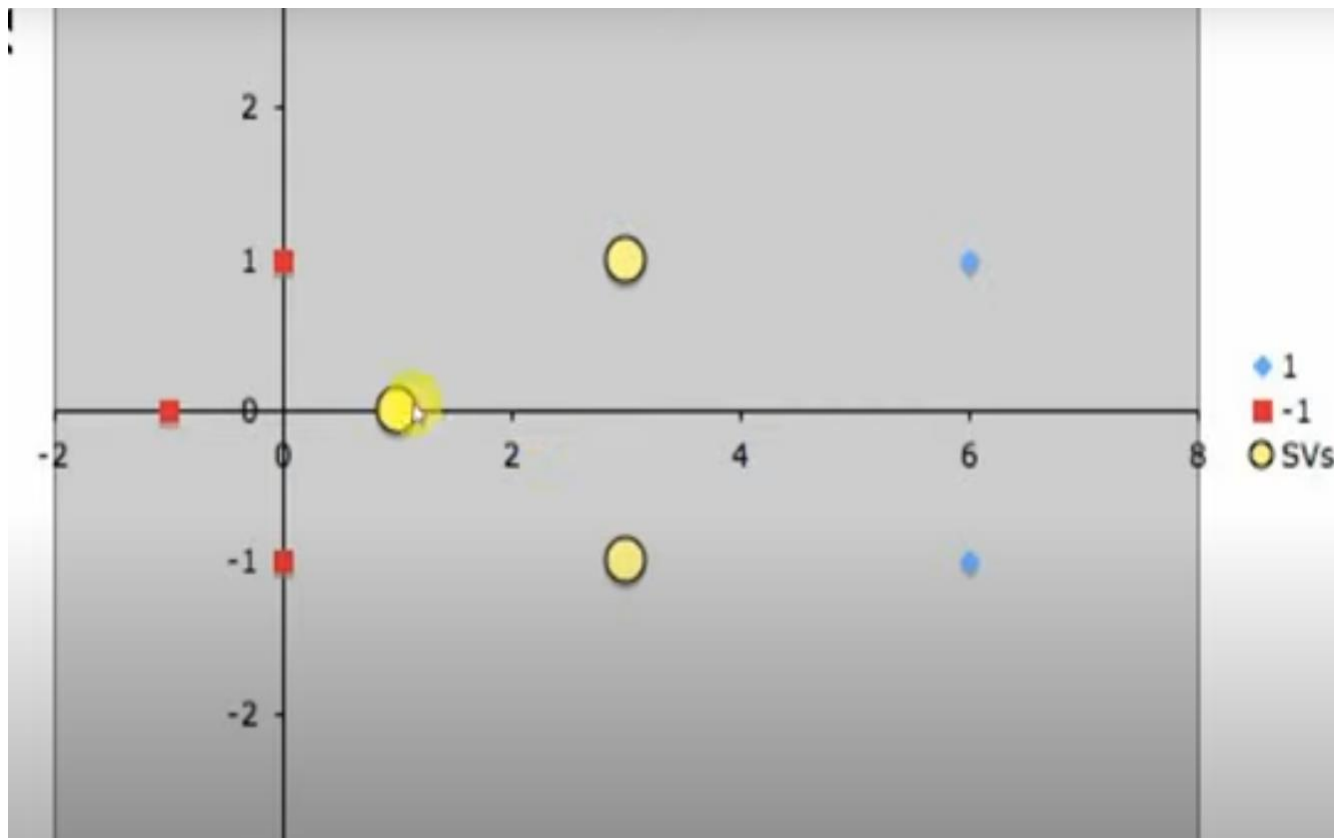


Support vectors:

- By inspection, it should be obvious that there are **three** support vectors,

$$\left\{ s_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, s_2 = \begin{pmatrix} 3 \\ 1 \end{pmatrix}, s_3 = \begin{pmatrix} 3 \\ -1 \end{pmatrix} \right\}$$

Illustration of support vectors (marked in yellow):



Use equalities for these support vectors:

- Negative $(1, 0)$, $y = -1$:

$$-(w_1 \cdot 1 + w_2 \cdot 0 + b) = 1 \Rightarrow -w_1 - b = 1.$$

- Positive $(3, 1)$, $y = +1$:

$$3w_1 + w_2 + b = 1.$$

- Positive $(3, -1)$, $y = +1$:

$$3w_1 - w_2 + b = 1.$$

Subtract the last two equalities:

$$(3w_1 + w_2 + b) - (3w_1 - w_2 + b) = 0 \Rightarrow 2w_2 = 0 \Rightarrow w_2 = 0.$$

With $w_2 = 0$, the positive equality gives $3w_1 + b = 1$. The negative equality gives $-w_1 - b = 1$. Add them:

$$(3w_1 + b) + (-w_1 - b) = 1 + 1 \Rightarrow 2w_1 = 2 \Rightarrow w_1 = 1.$$

Then $b = 1 - 3w_1 = 1 - 3 = -2$.

So

$w = (1, 0), \quad b = -2.$

- Decision boundary: $w \cdot x + b = x_1 - 2 = 0 \Rightarrow x_1 = 2$ (vertical line through $x = 2$).
- Check support equalities:
 - For $(1, 0)$, $y(w \cdot x + b) = -1 \cdot (1 - 2) = -1 \cdot (-1) = 1$.
 - For $(3, 1)$ and $(3, -1)$, $y(w \cdot x + b) = 1 \cdot (3 - 2) = 1$.
- All other points satisfy $y(w \cdot x + b) \geq 1$ (e.g. $(6, 1)$: $6 - 2 = 4 \geq 1$; $(0, 1)$: -2 times -1 gives $2 \geq 1$).

